СПбГЭТУ «ЛЭТИ»
ПЕРВЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ

МИКРОПРОЦЕССОРНАЯ ТЕХНИКА В МЕХАТРОНИКЕ И РОБОТОТЕХНИКЕ

Авторы:
Бельский Григорий Владимирович
Копычев Михаил Михайлович

Преподаватель:
Копычев Михаил Михайлович
Илатовская Екатерина Вадимовна

Санкт-Петербург 2025

1. Введение

Микроконтроллер (МК) – это микросхема, на кристалле которой построен микрокомпьютер. Это значит, что внутри единого корпуса собраны микросхемы процессора, памяти (ПЗУ и ОЗУ) и периферийных устройств. Все компоненты МК взаимодействуют друг с другом и внешними устройствами с помощью «микропрограммы», которая хранится внутри МК.

Основным назначением МК является управление различными электронными устройствами. С помощью МК можно управлять светодиодами, двигателями, считывать информацию с датчиков, работать с внешними переключателями и кнопками, проектировать системы управления, считывать аналоговые и цифровые сигналы, рисовать изображения на экранах, генерировать звук с помощью громкоговорителей или бузеров, считывать информацию с внешних устройства, организовать обмен данными между различными компонентами сложной системы по разным протоколам и т. д. Одним из основных отличий МК от микропроцессора является то, что устройства могут быть подключены непосредственно к МК, но не к микропроцессору.

STM32 – это семейство 32-битных МК, производства STMicroelectronics архитектуры ARM, представители этого семейства МК различаются производительностью, объемом памяти, типом корпуса и набором функциональных возможностей. На Рис. 1.1 показаны различные отладочные платы с указанием наименования центрального процессора (ЦП).

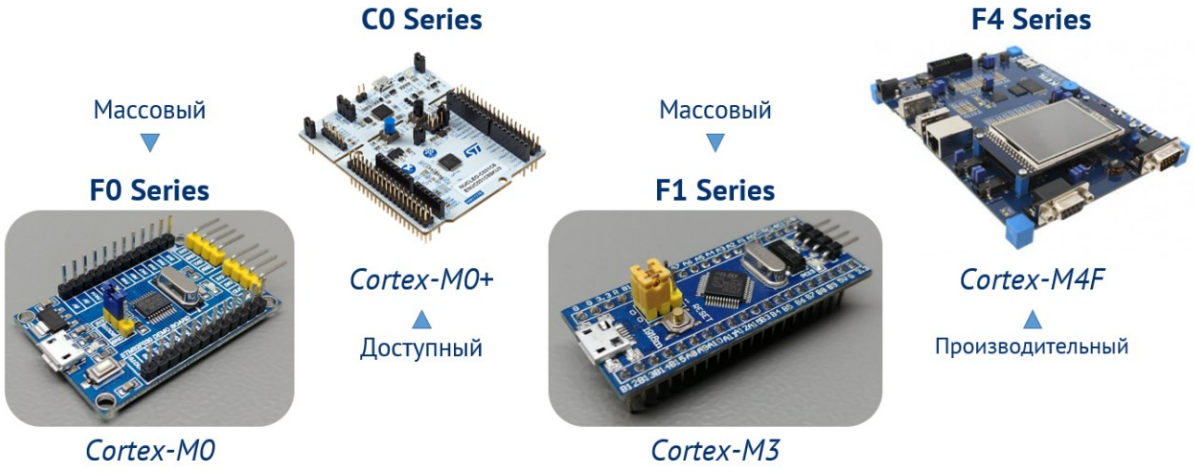


Рис. 1.1. Отладочные платы с МК STM32

Архитектура МК включает ЦП, пины (контакты) ввода/вывода, модули памяти, модуль внутрисистемного программирования, модуль прерываний, сторожевой таймер, таймеры/счетчики, аналого-цифровой преобразователь, аналоговый компаратор, модули протоколов (USART, I2C, SPI, USB) и др. Всё это делает МК подходящим инструментом для широкого спектра задач, начиная от RC-моделей и заканчивая системами управления, включающими в себя множество установок и протоколов обмена данными. Крошечная часть областей применения МК показана на Рис. 1.2.

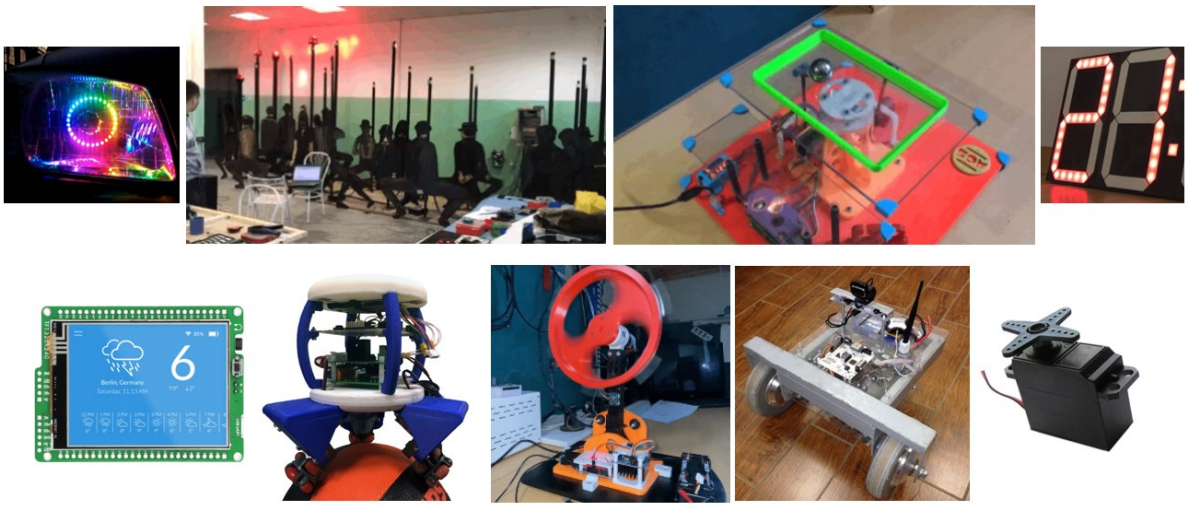


Рис. 1.2. Области применения МК

На Рис. 1.2 приведены следующие приложения МК (сверху вниз и слева направо): управление светодиодами, управление двигателями с обратными связями, система управления стабилизацией неустойчивого объекта на платформе, управление таймером на семисегментных индикаторах, ЖК-экран с прогнозом погоды, управление стабилизацией робота «ballbot», система управления положением перевёрнутого маятника, робот с двумя приводными колесами на BLDC моторах и телеуправлением, управление серводвигателем (сервомашинкой).

Рассмотрим подробнее конкретный МК STM32F401xC на примере отладочной платы Black Pill, распиновка которой показана на Рис. 1.3.

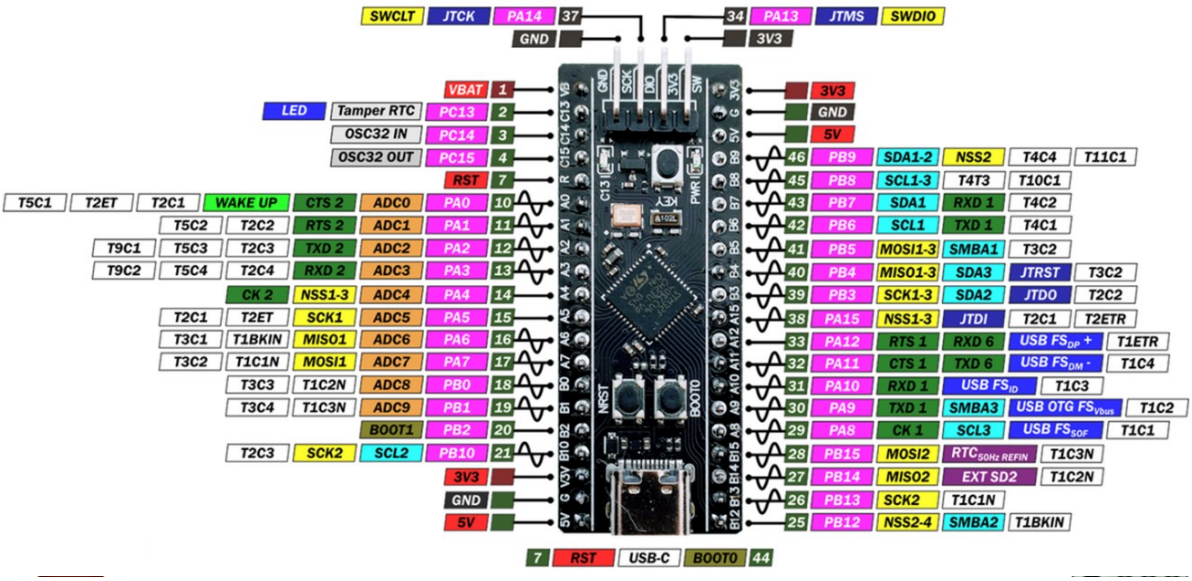


Рис. 1.3. Распиновка отладочной платы Black Pill с МК STM32F401xC

Каждый МК имеет разное количество контактов ввода-вывода, в нашем случае контактов – 36. На плате также выведены контакты питания (итого выводов – 48). МК не имеет операционной системы, поэтому пользователь может контролировать напряжение на выводах МК напрямую с помощью

программного кода. Внешние устройства могут быть подключены к контактам МК. Для адресации этих выводов в программе, они объединены в порты. Порт – это комбинация выводов МК, каждый порт имеет имя и это имя представляет собой букву латинского алфавита, например: порт *A*, порт *B* и т.д. В распиновке МК пины порта отмечены буквами *Pxn*, где *x* – имя порта (буква), *n* – номер пина в порту (по умолчанию от 0 до 15). Важно, что практически каждый вывод МК имеет альтернативные функции. Например, вывод 46 (PB9) в дополнение к основной функции имеет функции вывода ШИМ таймеров 4 и 11, а также является выводом протоколов I2C и SPI.

STM32F401xC имеет 256 Кб перепрограммируемой флэш-памяти, 64 Кб ОЗУ, семь таймеров общего назначения и один усовершенствованный таймер, 24 канала ШИМ, 12-битный АЦП с 16-ю каналами, по три модуля протоколов I2C, USART и SPI и т.д.

Каждый МК должен работать на требуемой частоте. Частота может быть задана либо от внутреннего источника МК, либо внешним устройством, также есть вариант увеличить частоту тактирования ядра до (в случае МК STM32F401xC) 84 МГц.

Для удобства программирования МК STM32 неплохо иметь в непосредственном доступе несколько документов, скачиваемых с официального сайта STMicroelectronics, а именно:

- Datasheet (ds) или руководство по эксплуатации на конкретный МК, номер документа начинается с букв DS (в случае МК STM32F401xC – DS9716) – в нем приведена базовая информация о МК, в частности – структура МК, список альтернативных функций пинов ввода/вывода и прочее;
- Programming manual (pm) или руководство по программированию, номер документа начинается с букв PM (в случае МК STM32F401xC – PM0214) – в нем приведена информация о ЦП, в частности – регистры для работы с базовым таймером SysTick и прочей периферией непосредственно относящейся к ЦП;
- Reference manual (rm) или справочное руководство, номер документа начинается с букв RM (в случае МК STM32F401xC – RM0368) – в нем приведена информация обо всех регистрах ввода/вывода;
- Если применяется готовые решения, вроде отладочных плат Pill или Nucleo, нелишним будет иметь схему платы (schematic) и руководство (user manual) к ней, номер документа начинается с букв UM (в случае МК STM32F401xC и платы Black Pill – UM3461).

1.1. Сравнение AVR и STM32

Основные различия архитектур сведены в Табл. 1.1.

Таблица 1.1 – Сравнение AVR и STM32

Характеристика	AVR	STM32
Разрядность	8 бит	32 бита
Тактовая частота	16-20 МГц	до 400 МГц и выше
Архитектура	Гарвардская	Модифицированная Гарвардская
Тактирование	Простое, постоянные источники	Сложное, гибкое (PLL, предделители)
Организация памяти	Единое адресное пространство	Иерархическая (Flash, RAM, периферия)
Работа с периферией	Отдельные регистры	Memory-mapped регистры
Инструменты разработки	Atmel Studio, avr-gcc	STM32CubeIDE, Keil, IAR

Ключевые отличия в программировании:

```
// настройка вывода в AVR
    DDRB |= (1 << DDRB5); // направление данных на PB5
    PORTB |= (1 << PORTB5); // установка уровня напряжения

// настройка вывода в STM32
    // включить тактирование GPIOB
    RCC->AHB1ENR |= (1 << RCC_AHB1ENR_GPIOBEN_Pos);
    // направление данных на PB5
    GPIOB->MODER |= (1 << GPIO_MODER_MODE5_Pos);
    // установка уровня напряжения
    GPIOB->ODR |= (1 << GPIO_ODR_OD5_Pos);
```

Преимущества STM32:

МК STM32 сочетают высокую производительность с богатым набором периферии. Их 32-битные ядра, такие как Cortex-M4 с поддержкой операций с плавающей запятой, работают на частотах от 84 МГц и выше, обеспечивая эффективную обработку данных. Архитектура с конвейеризацией и специализированными DSP-инструкциями позволяет выполнять сложные вычисления и алгоритмы цифровой обработки сигналов.

Богатство встроенной периферии — ключевое достоинство. Контроллеры предлагают множество интерфейсов связи (SPI, I2C, USART, USB, CAN, Ethernet), мощные таймеры для управления приводами, а также аналоговые модули, включая АЦП и ЦАП. Передачу данных между этими

модулями разгружают контроллеры прямого доступа к памяти (DMA), высвобождая ресурсы ядра.

Платформа отличается гибкостью в управлении энергопотреблением. Доступны несколько режимов энергосбережения — от Sleep до глубокого Standby с током порядка микроампер. Экономия достигается также за счёт динамического изменения тактовой частоты и индивидуального отключения тактирования неиспользуемых модулей.

Разработку существенно упрощает мощная экосистема: бесплатная среда STM32CubeIDE, инструмент графической конфигурации STM32CubeMX, а также библиотеки HAL и LL. Проекты поддерживаются обширной документацией и активным сообществом.

Почему переход на STM32 оправдан?

Использование STM32 открывает возможности для реализации сложных проектов, которые не по силам 8-битным контроллерам. Богатая периферия позволяет уменьшить количество внешних компонентов, снижая стоимость и сложность платы. Выбор в пользу 32-битной архитектуры обеспечивает запас для будущего развития продукта и предлагает масштабируемость в рамках единого семейства.

Сложности при переходе

Основные трудности связаны с начальным этапом. Новым разработчикам требуется время, чтобы освоить более сложную систему тактирования и тонкой настройки периферии. Объём документации, хотя и является преимуществом, поначалу может быть перегруженным, а процесс первоначальной конфигурации проекта включает больше шагов, чем для более простых МК.

2. НАСТРОЙКА ПРОЕКТА В **STM32CUBEIDE**

Для написания программ будем использовать IDE непосредственно от разработчиков МК STMicroelectronics – STM32CubeIDE.

После запуска STM32CubeIDE (далее – CubeIDE) может появиться диалоговое окно, в котором пользователю предлагается выбрать рабочее пространство (workspace) (см. Рис. 2.1) – это папка, в которой по умолчанию будут создаваться все проекты. Создайте рабочее пространство в подходящем месте на жестком диске вашего компьютера.

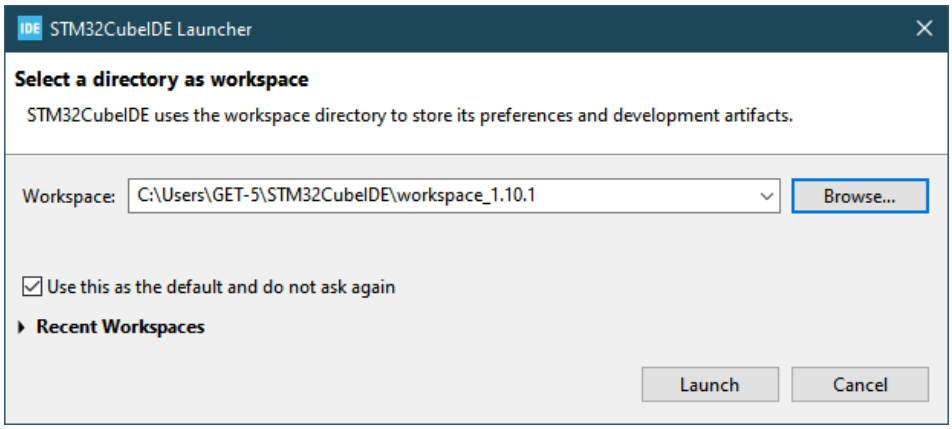


Рис. 2.1. Указание рабочего пространства workspace

В следующем окне нужно выбрать *File -> New -> STM32 Project Create/Import* и выбрать *Empty Project*, как это показано на Рис. 2.2.

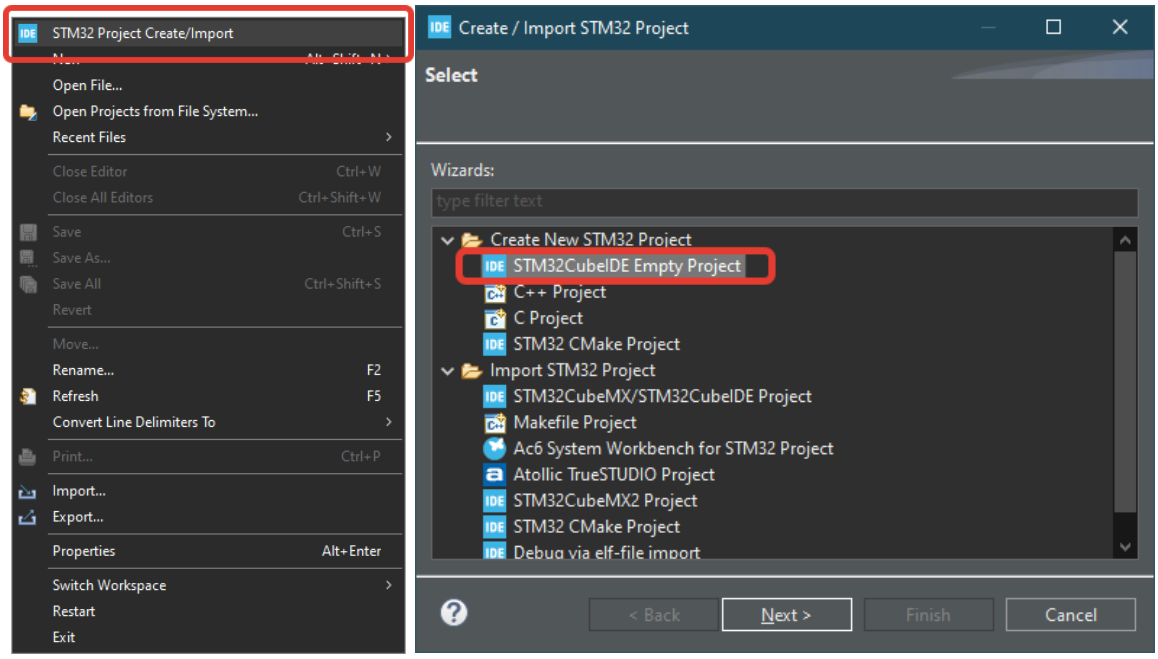


Рис. 2.2. Создание нового проекта STM32 Project

В следующем окне пользователю предлагается выбрать под какой именно МК создаётся проект. В нашем случае МК носит имя **STM32F401CCU6**. Это название можно ввести в поле *Mcu Name Filter* и далее в поле справа выбрать нужный нам МК, как это показано на Рис. 2.3.

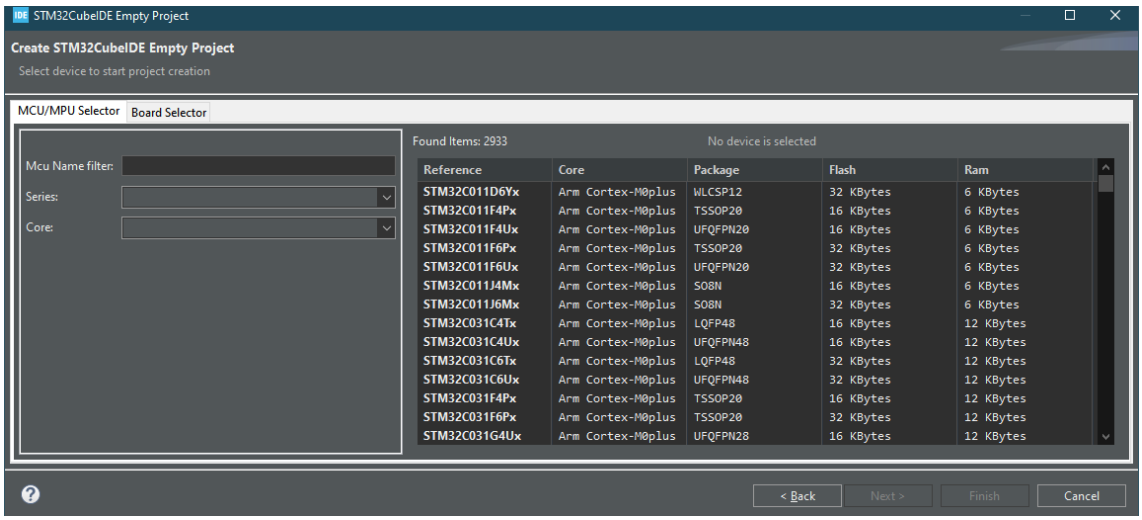


Рис. 2.3. Выбор программируемого МК STM32F401CCU6

После этого нужно нажать кнопку *Next* в нижней части окна. В следующем окне нужно ввести имя вашего проекта в поле *Project Name*, как это показано на Рис. 2.4. Остальные поля можно оставить без изменений.

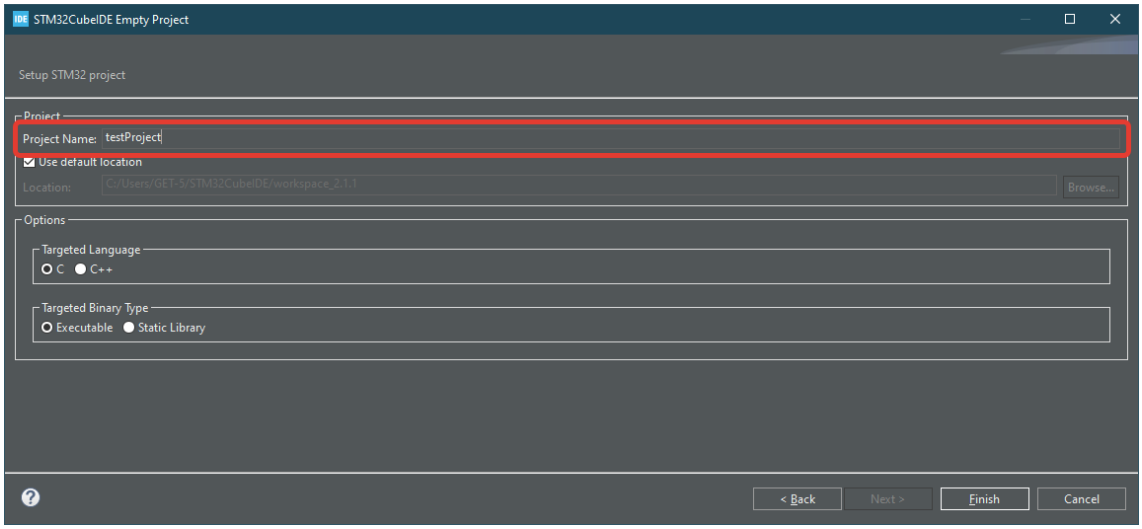


Рис. 2.4. Ввод названия проекта и выбор Targeted Project Type – Empty

После этого следует нажать на кнопку Finish и пользователю будет представлено рабочее окно среды разработки, в котором слева находится дерево проекта Project Explorer, а справа – пространство для работы с файлами.

Прежде чем писать код, следует подготовить среду разработки для взаимодействия с определённым МК, то есть добавить в проект соответствующие МК заголовочные файлы, а также заголовки CMSIS (Common Microcontroller Software Interface Standard), позволяющим программно взаимодействовать с МК посредством встроенных в него регистров.

На Рис. 2.5.1 и 2.5.2 приведены два дерева проекта, причем на Рис. 2.5.1 приведено изначальное дерево проекта, а на Рис. 2.5.2 – необходимый вид

дерева проекта, со всеми заголовочными файлами, скопированными в папку *Inc* в *Project Explorer*. Все файлы будут предоставлены.

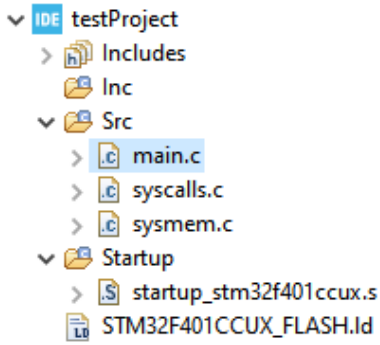


Рис. 2.5.1. Изначальное содержание

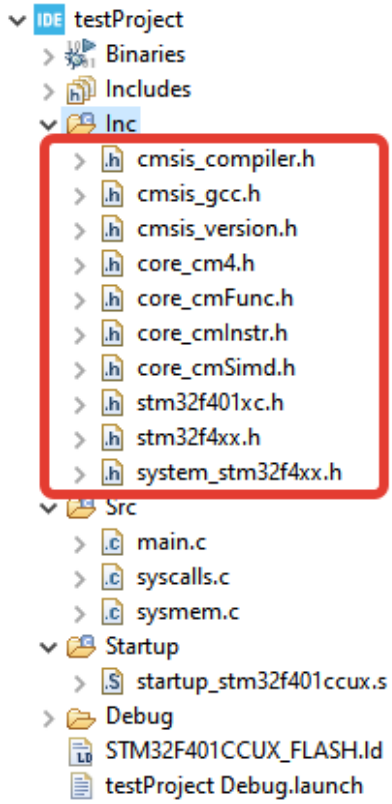


Рис. 2.5.2. Требуемое содержание

После копирования файлов в папку *Inc*, следует перейти в свежескопированный файл stm32f4xx.h и в нём найти область @brief STM32 Family и в списке #define снять комментарий со строки #define STRM32F401xC (см. Рис. 2.6).

```
/**
  * @brief STM32 Family
  */
#if !defined  (STM32F4)
#define STM32F4
#endif /* STM32F4 */

/* Uncomment the line below according to the target STM32 device used in your
   application
  */
#if !defined (STM32F405xx) && !defined (STM32F415xx) && !defined (STM32F407xx) && !defined (STM32F417xx) && \
    !defined (STM32F427xx) && !defined (STM32F437xx) && !defined (STM32F429xx) && !defined (STM32F439xx) && \
    !defined (STM32F401xC) && !defined (STM32F401xE) && !defined (STM32F410Tx) && !defined (STM32F410Cx) && \
    !defined (STM32F410Rx) && !defined (STM32F411xE) && !defined (STM32F446xx) && !defined (STM32F469xx) && \
    !defined (STM32F479xx) && !defined (STM32F412Cx) && !defined (STM32F412Rx) && !defined (STM32F412Vx) && \
    !defined (STM32F412Zx) && !defined (STM32F413xx) && !defined (STM32F423xx)
/* #define STM32F405xx */   /*!< STM32F405RG, STM32F405VG and STM32F405ZG Devices */
/* #define STM32F415xx */   /*!< STM32F415RG, STM32F415VG and STM32F415ZG Devices */
/* #define STM32F407xx */   /*!< STM32F407VG, STM32F407VE, STM32F407ZG, STM32F407ZE, STM32F407IG  and STM32F407IE Devices */
/* #define STM32F417xx */   /*!< STM32F417VG, STM32F417VE, STM32F417ZG, STM32F417ZE, STM32F417IG and STM32F417IE Devices */
/* #define STM32F427xx */   /*!< STM32F427VG, STM32F427VI, STM32F427ZG, STM32F427ZI, STM32F427IG and STM32F427II Devices */
/* #define STM32F437xx */   /*!< STM32F437VG, STM32F437VI, STM32F437ZG, STM32F437ZI, STM32F437IG and STM32F437II Devices */
/* #define STM32F429xx */   /*!< STM32F429VG, STM32F429VI, STM32F429ZG, STM32F429ZI, STM32F429BG, STM32F429BI, STM32F429NG,
                               STM32F439NI, STM32F429IG  and STM32F429II Devices */
/* #define STM32F439xx */   /*!< STM32F439VG, STM32F439VI, STM32F439ZG, STM32F439ZI, STM32F439BG, STM32F439BI, STM32F439NG,
                               STM32F439NI, STM32F439IG and STM32F439II Devices */
#define STM32F401xC    /*!< STM32F401CB, STM32F401CC, STM32F401RB, STM32F401RC, STM32F401VB and STM32F401VC Devices */
/* #define STM32F401xE */   /*!< STM32F401CD, STM32F401RD, STM32F401VD, STM32F401CE, STM32F401RE and STM32F401VE Devices */
/* #define STM32F410Tx */   /*!< STM32F410T8 and STM32F410TB Devices */
```

Рис. 2.6. Снимите комментарий со строки #define STM32F401xC в файле stm32f4xx.h

На этом настройка заканчивается и можно перейти непосредственно к написанию кода. Для каждого МК набор файлов CMSIS будет различным.

3. Первая программа

В начале main() следует провести инициализацию тактирования МК. Делается это с помощью модуля RCC (Reset and Clock Control). Для лучшего понимания обратимся к Рис. 3.1.

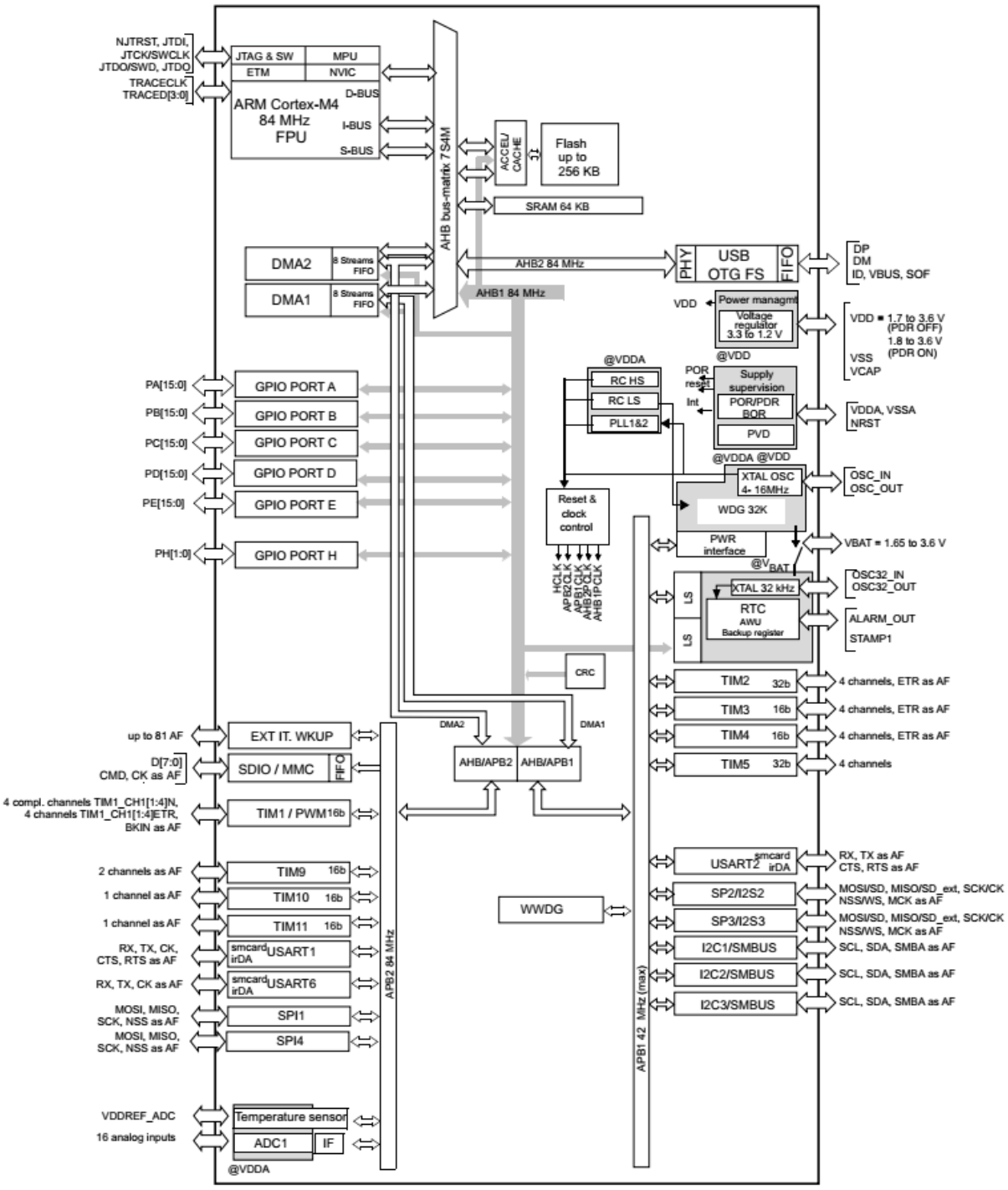


Рис. 3.1. Структурная схема МК STM32F401xC

На Рис. 3.1 в левом верхнем углу изображён ЦП ARM Cortex-M4 (в случае другого МК ЦП, разумеется, может быть другой), который связывается с периферией через шину AHB (Advanced High-Performance Bus). Чуть ниже ЦП, также по левой стороне можно увидеть модуль GPIO, представленный портами ввода-вывода, например, GPIOA, GPIOB, GPIOC и т.д. Порты ввода-

вывода связаны с ЦП через шину AHB1. Если посмотреть ещё ниже, то можно увидеть модули таймеров (TIM1 / PWM, TIM9, TIM10, …), модули UART (USART1 и USART6), модули SPI (SPI1 и SPI4) и АЦП (ADC1), которые подключены к шине APB2 (Advanced Peripheral Bus). С правой стороны можно наблюдать подключённые к шине APB1 ещё модули таймеров, а также модули протоколов UART, SPI, I2C. Следует обратить внимание, что шины APB1 и APB2 имеют неодинаковую предельную частоту тактирования. Для того, чтобы программно взаимодействовать с некоторым модулем МК, нужно предварительно включить его тактирование на соответствующей шине через RCC.

На Рис. 3.2 приведена схема тактирования (Clock Tree) МК.

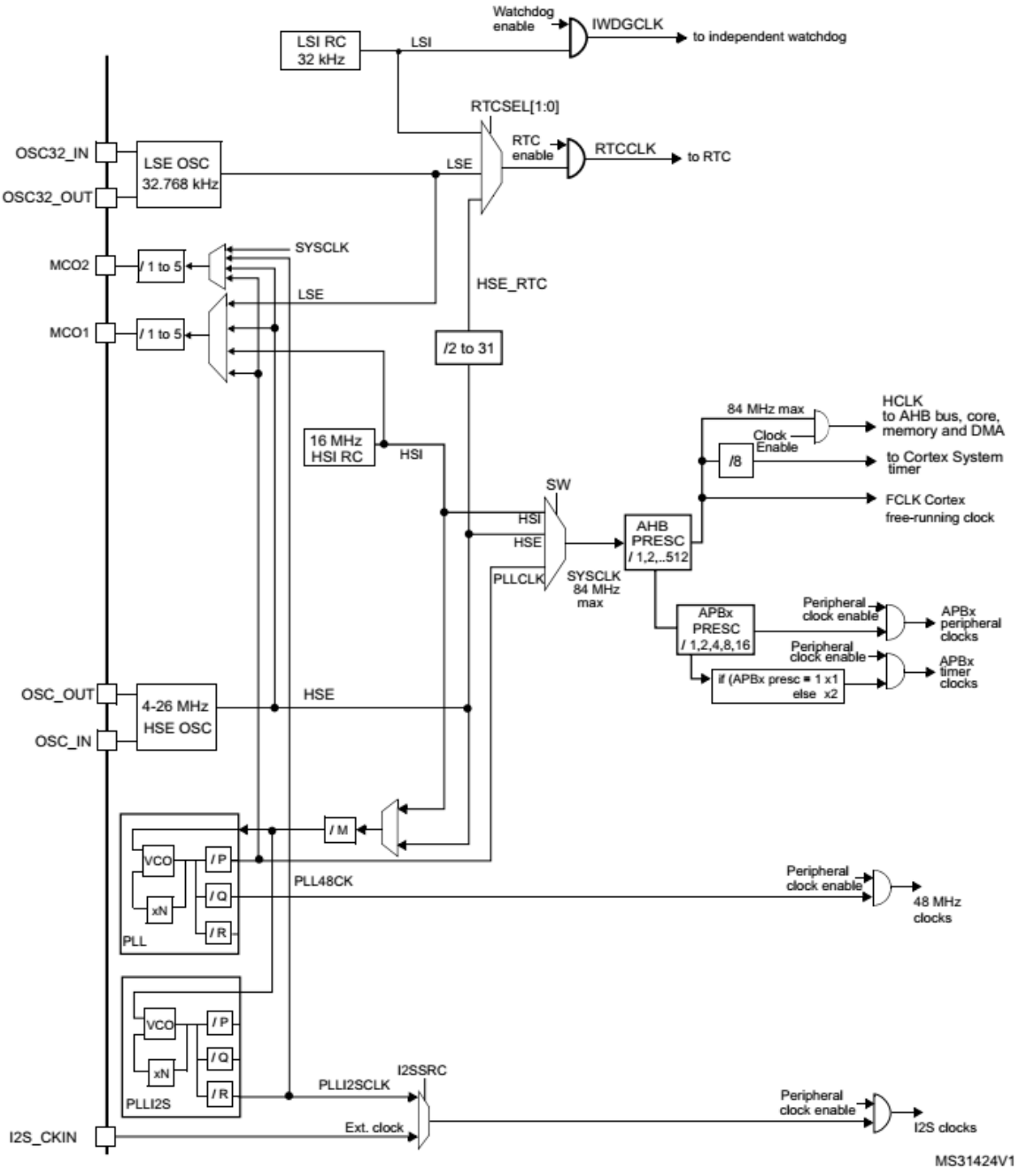


Рис. 3.2. Схема тактирования МК STM32F401xC

Слева на Рис. 3.2 расположены выводы МК, а примерно посередине расположен SYSCLK (главное системное тактирование), который задаёт частоту шинам APB*x* и AHB. Существует три источника тактирующего сигнала для SYSCLK:

- Внутренний резонатор HSI (High-Speed Internal) – 16 МГц;
- Внешний резонатор HSE (High-Speed External) – 4…26 МГц;
- Фазовая автоподстройка частоты PLL (Phase-Lock Loop) – до 84 МГц.

Выбор источника тактирования можно сделать кодом в самом начале программы путём записи значений в соответствующие регистры RCC. Здесь рассмотрим только самые необходимые, но на самом деле настроек больше и все их можно посмотреть в Reference manual.

Чтобы выбрать задатчик тактирующего сигнала нужно обратиться к регистру RCC clock control register (RCC_CR).

Регистр **RCC_CR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				PLLI2S RDY	PLLI2S ON	PLLRDY	PLLON	Reserved				CSS ON	HSE BYP	HSE RDY	HSE ON
				r	rw	r	rw					rw	rw	r	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
HSICAL[7:0]								HSITRIM[4:0]					Res.	HSI RDY	HSION
r	r	r	r	r	r	r	r	rw	rw	rw	rw	rw		r	rw

Бит HSE ON включает и отключает внешнее тактирование, HSI ON, соответственно – внутреннее тактирование. Здесь же можно включить PLL, но настройка PLL несколько сложнее, чем просто выставление бита PLL ON.

Включение соответствующего тактирования запускает именно сам процесс тактирования, то есть возникновение импульсов напряжения, его использование настраивается отдельно. Запуск тактирования происходит не мгновенно. Успешный запуск подтверждается установкой флага RDY для каждого из источников: HSE – HSE RDY, HSI – HSI RDY, PLL – PLL RDY.

После выбора и запуска задатчика тактирующего сигнала следует выбрать этот тактирующий сигнал в качестве системного. Делается это с помощью регистра RCC clock configuration register (RCC_CFGR).

Регистр **RCC_CFGR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MCO2		MCO2 PRE[2:0]			MCO1 PRE[2:0]			I2SSCR	MCO1		RTCPRE[4:0]				
rw		rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PPRE2[2:0]			PPRE1[2:0]			Reserved		HPRE[3:0]				SWS1	SWS0	SW1	SW0
rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	r	r	rw	rw

Пара битов SW0 и SW1 позволяют задать источник системного тактирования, а именно SW[1:0]: 00 – выбор HSI, 01 – выбор HSE, 10 – выбор PLL, 11 – зарезервированная комбинация. Биты HPRE[3:0], PPRE1[2:0] и

PPRE2[2:0] задают предделители (относительно SYSCLK) частот шин AHB, APB1 и APB2, соответственно (это требуется, если частота SYSCLK высока и, например, шина APB1 не может работать на такой высокой частоте), а именно HPRE[3:0]: 0xxx – предделитель на 1, 1000 – 2, 1001 – 4, 1010 – 8, 1011 – 16, 1100 – 64, 1101 – 128, 1110 – 256, 1111 – 512; PPRE1[2:0] и PPRE2[2:0]: 0xxx – предделитель на 1, 100 – 2, 101 – 4, 110 – 8, 111 – 16.

После включения и выбора задатчика частоты, можно воспользоваться одним из периферийных модулей, самым очевидным из которых является модуль портов ввода-вывода GPIO. Прежде чем воспользоваться модулем, его следует включить, а точнее – включить его тактирование. Поскольку GPIO модуль подключен к шине AHB (см. Рис. 3.1), то и включение его тактирования будет производится с помощью регистра RCC AHB1 peripheral clock enable register (RCC_AHB1ENR).

Регистр **RCC_AHB1ENR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									DMA2EN	DMA1EN	Reserved				
									rw	rw					

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CRCEN	Reserved				GPIOH EN	Reserved		GPIOEEN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
			rw					rw			rw	rw	rw	rw	rw

На каждой плате Black Pill есть светодиод, подключенной к выводу PC13, как это показано на Рис. 3.3, поэтому следует включить тактирование порта C выставив бит GPIOCEN регистра RCC_AHB1ENR.

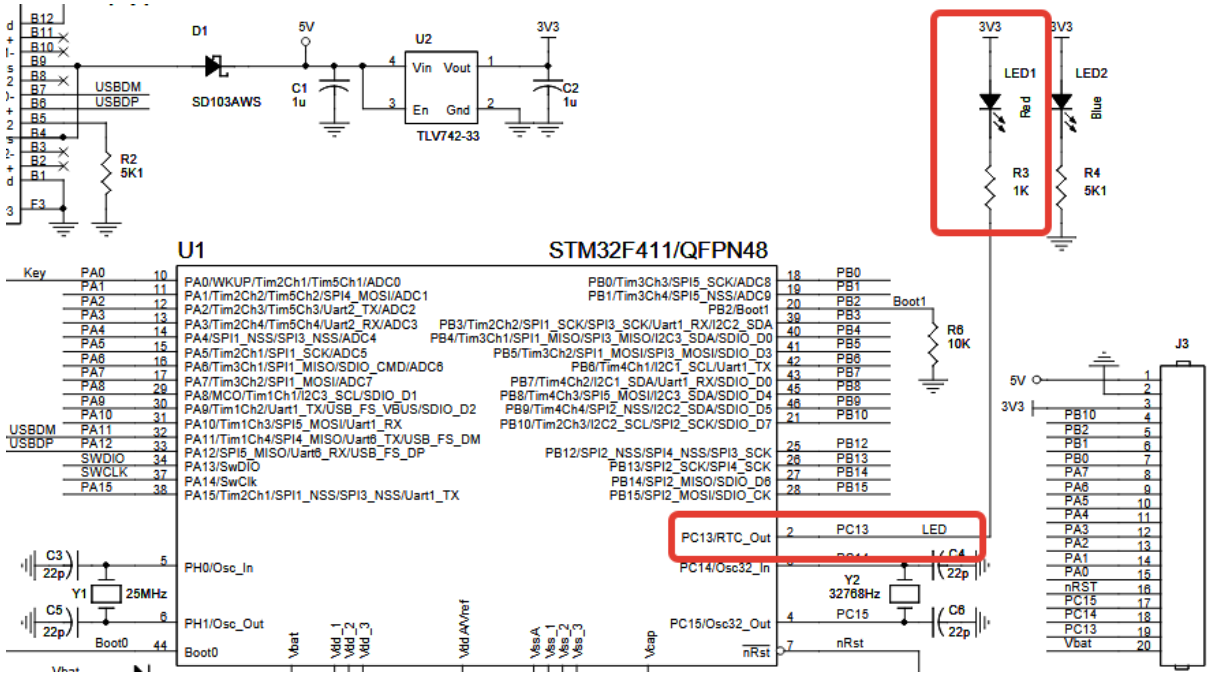


Рис. 3.3. Фрагмент электрической схемы платы Black Pill

После включения тактирования порта следует настроить вывод PC13 на выход для того, чтобы иметь возможность посылать с него логические уровни напряжения. Это можно сделать с помощью регистра GPIO port mode register

(GPIO*x*_MODER), а именно пары бит MODER*i*[1:0], где *i* – это номер вывода внутри порта: 00 – вывод настроен как вход (начальное значение), 01 – вывод настроен как выход, 10 – вывод настроен на альтернативную функцию, 11 – вывод настроен на аналоговый режим.

Регистр **GPIO*x*_MODER**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

После настройки вывода МК на режим выхода, логический уровень напряжения, который с него пойдёт по умолчанию – это логический ноль. Чтобы изменить логический уровень напряжения на выводе МК, следует обратиться к регистру GPIO port output data register (GPIO*x*_ODR), а именно к отдельным битам ODR*i*: 0 – на выводе *i* соответствующего порта будет наведён логический ноль (низкий уровень напряжения), 1 – на выводе *i* соответствующего порта будет наведена логическая единица (высокий уровень напряжения).

Регистр **GPIO*x*_ODR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Таким образом, функция main(), в которой производится попеременное включение и отключение светодиода на выводе PC13 будет выглядеть так, как это показано ниже.

```
#include <stdint.h> // заголовочный файл с типами данных
#include "stm32f401xc.h" // заголовочный файл с регистрами

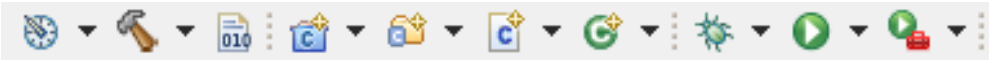
int main(void) {

    // включить тактирование от внешнего кварцевого резонатора
    RCC->CR |= (1 << RCC_CR_HSEON_Pos);
    // дождаться стабилизации тактирующего сигнала
    while (!(RCC->CR & (1 << RCC_CR_HSERDY_Pos))) {
        // «пустая» инструкция
        __NOP();
    }
    // выбрать HSE в качестве SYSCLK
    RCC->CFGR |= (1 << RCC_CFGR_SW_Pos);
    // включить тактирование GPIOC
    RCC->AHB1ENR |= (1 << RCC_AHB1ENR_GPIOCEN_Pos);
    // настроить вывод PC13 как выход МК
    GPIOC->MODER |= (1 << GPIO_MODER_MODE13_Pos);
    // бесконечный цикл
```

```
    for (;;) {
        // выставить логическую 1 на выводе PC13
        GPIOC->ODR |= (1 << GPIO_ODR_OD13_Pos);
        for (uint32_t i = 0; i < 500000; i++) {
            __NOP(); // задержка 500000 тактов
        }
        // выставить логический 0 на выводе PC13
        GPIOC->ODR &= ~(1 << GPIO_ODR_OD13_Pos);
        for (uint32_t i = 0; i < 500000; i++) {
            __NOP(); // задержка 500000 тактов
        }
    }
}
```

В Таблице 3.1 приведены некоторые функциональные кнопки панели управления CubeIDE.

Таблица 3.1 – Функциональные кнопки CubeIDE



Кнопка	Функционал	Описание
	Сборка	Проверка кода на наличие предупреждений и ошибок и сборка всех файлов
	Отладка	Запуск дебагера для построчного исполнения кода и чтения значения в регистрах
	Прошивка	Запись кода на МК через подключённый к компьютеру программатор ST-LINK/V2

После сборки проекта и прошивки МК светодиод на плате будет моргать с периодом, равным примерно одной секунде. После каждого изменения в коде не забывайте пересобирать проект перед прошивкой МК (в настройках прошивки Run Configurations можно выставить отметку Enable auto build на вкладке Main и тогда пересобираться проект будет автоматически при прошивке).

На картинках с содержимым регистров под именем каждого бита можно заметить различные способы доступа к этим битам. Приведём здесь варианты:

- **r** – только чтение;
- **w** – только запись;
- **rw** – чтение или запись;
- **rc_w0** – чтение или сброс записью 0 (запись 1 игнорируется);
- **rc_w1** – чтение или сброс записью 1 (запись 0 игнорируется);
- **rc_r** – при чтении выполняется автоматический сброс бита;
- **rs** – чтение или запись единицы (запись 0 игнорируется);
- **rt_w** – чтение или запись, при записи значение не меняется, но это триггер для какого-то события.

Схема лабораторного стенда приведена на Рис. 3.4.

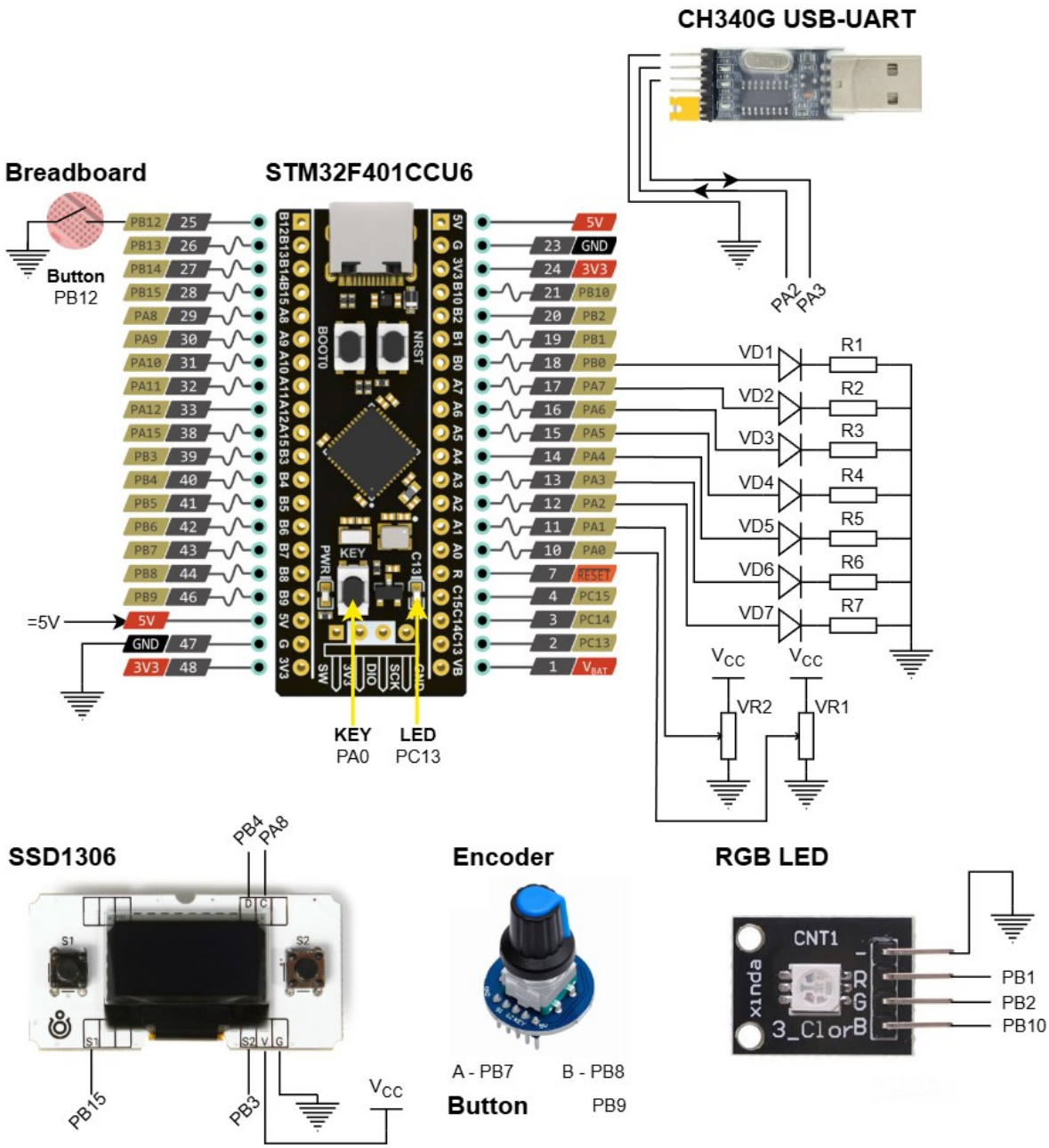


Рис. 3.4. Схема лабораторного стенда

При выполнении лабораторных работ желательно хотя бы один раз за работу ознакамливаться со схемой на Рис.3.4.

Содержание отчета:

- титульный лист с подписью студента, выполнившего работу;
- цель работы своими словами;
- электрическая схема лабораторной работы;
- блок-схема программы;
- код программы по количеству заданий в работе с комментариями;
- скриншот дерева проекта, с папками Inc и Src (проект должен быть разделён на несколько **.h** и **.c** файлов);
- выводы по работе своими словами.

4. Лабораторные работы

Работа 1. Программирование реверсивного "бегущего огня" с использованием GPIO, системного таймера SysTick и обработки внешних прерываний

Электрическая схема к лабораторной работе приведена на Рис. 4.1.1.

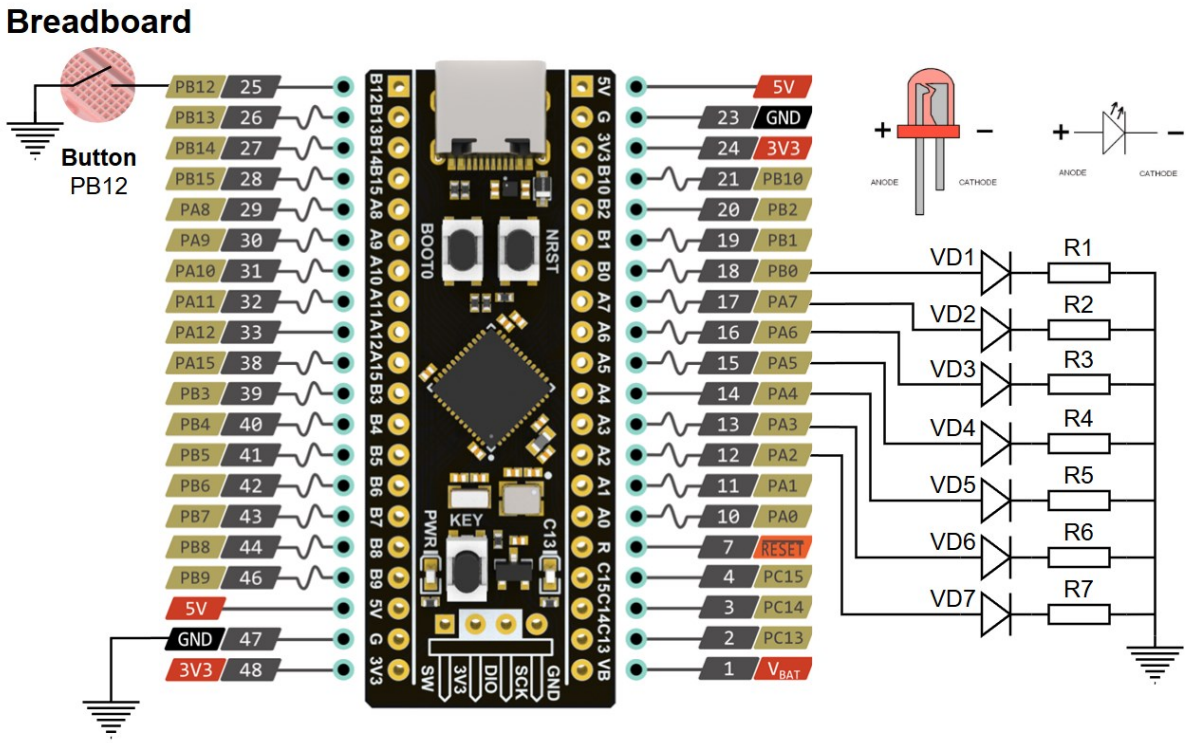


Рис. 4.1.1. Схема к лабораторной работе №1

К МК подключены 7 светодиодов VD*i*, каждый подведён анодом к выводу МК. Катоды светодиодов подведены к GND (справа внизу на Рис. 4.1.1) и соединены с GND МК (слева внизу на Рис. 4.1.1). Также в работе будет принимать участие тактовая кнопка, расположенная на макетной плате, находящейся сбоку от Black Pill. На Рис. 4.1.1. эта кнопка подписана как Button и подключена к выводу PB12 МК (см. верхний левый угол Рис. 4.1.1).

Задание на работу №1

напишите программу для последовательного зажигания семи светодиодов VD*i* (см. Рис. 4.1.1); светодиоды переключаются каждые 0.25 секунды (250 мс); направление последовательности зажигания светодиодов – либо VD1 → VD7, либо VD7 → VD1 – задаётся нажатием на кнопку Button (вывод PB12 МК).

Программа должна сохранять работоспособность при настройке частоты тактирования МК на HSI, HSE и PLL.

Сведём предлагаемую задачу к виду алгоритма, приведённого на Рис. 4.1.2.



Рис. 4.1.2. Алгоритмическая блок-схема к работе №1

Задание можно разделить на две части – часть с зажиганием светодиодов и часть с кнопкой. Часть работы с зажиганием светодиодов уже была затронута в предыдущей главе, а именно – работа с регистрами настройки выводов МК на вход или выход (регистр GPIO*x*_MODER) и подача логического уровня напряжения с выводов МК, настроенных как выходы (регистр GPIO*x*_ODR). В лабораторной работе рекомендуется использовать регистр GPIO*x*_BSRR.

Если обратить внимание на Рис. 4.1.2, то выяснится, что диоды зажигаются через паузу (горят в течение этой паузы). Паузу можно организовать также, как это было описано в главе 3 настоящего пособия, то есть через пустой цикл с инструкцией `__NOP();` внутри. Однако, лучше организовать функцию задержки с помощью простого системного таймера, который встроен в ЦП МК и этот таймер называется SysTick (таймер описывается в документе Programming manual).

Системный таймер имеет свои регистры. Рассмотрим регистр управления SysTick control and status register (SysTick_CTRL), а именно: бит ENABLE включает (логическая единица) или отключает (логический ноль) таймер, бит TICKINT разрешает (1) или запрещает (0) прерывание от таймера, бит CLKSOURCE позволяет выбрать в качестве источника тактирования либо AHB (1), либо AHB/8 (0), бит COUNTFLAG принимает значение 1, если таймер досчитал от начального до конечного значения.

Регистр **SysTick_CTRL**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															COUNT FLAG
															rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													CLKSO URCE	TICK INT	EN ABLE
													rw	rw	rw

Следующий регистр – SysTick reload value register (SysTick_LOAD). Этот регистр хранит начальное счётное значение таймера, то есть значение, с которого таймер начинает считать и после того, как таймер дойдёт до нуля, таймер опять начинает считать со значения SysTick_LOAD. Значение счётного регистра изменяется с частотой тактирования AHB/8 (если не выбрано иное в регистре SysTick_CTRL). Пусть частота AHB составляет 25 МГц, значит за одну секунду значение в счётном регистре должно измениться с 3 125 000 до 0. В 24-битный регистр SysTick_LOAD можно положить любое значение до $(2^{24} – 1)$. Отсюда значит, что за 1 мс таймер досчитает от 3125 до 0. И вот число (3125 – 1) нужно поместить в SysTick_LOAD.

Регистр **SysTick_LOAD**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								RELOAD[23:16]							
								rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RELOAD[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Алгоритм функции задержки в миллисекундах (прототип **`void delayMs(uint32_t millisec)`**) приведён на Рис. 4.1.3. Функция принимает в себя число, равное величине задержки. Таким образом ранее введённый пустой цикл, рассмотренный в главе 3 настоящего пособия, заменяется на одну строку, а именно:

```
int main(void) {
	// TODO
	for (;;) {
		// попеременное включение и отключение диода на PC13
		GPIOC->ODR ^= (1 << GPIO_ODR_OD13_Pos);
		// задержка 500 миллисекунд
		delayMs(500);
	}
}
```

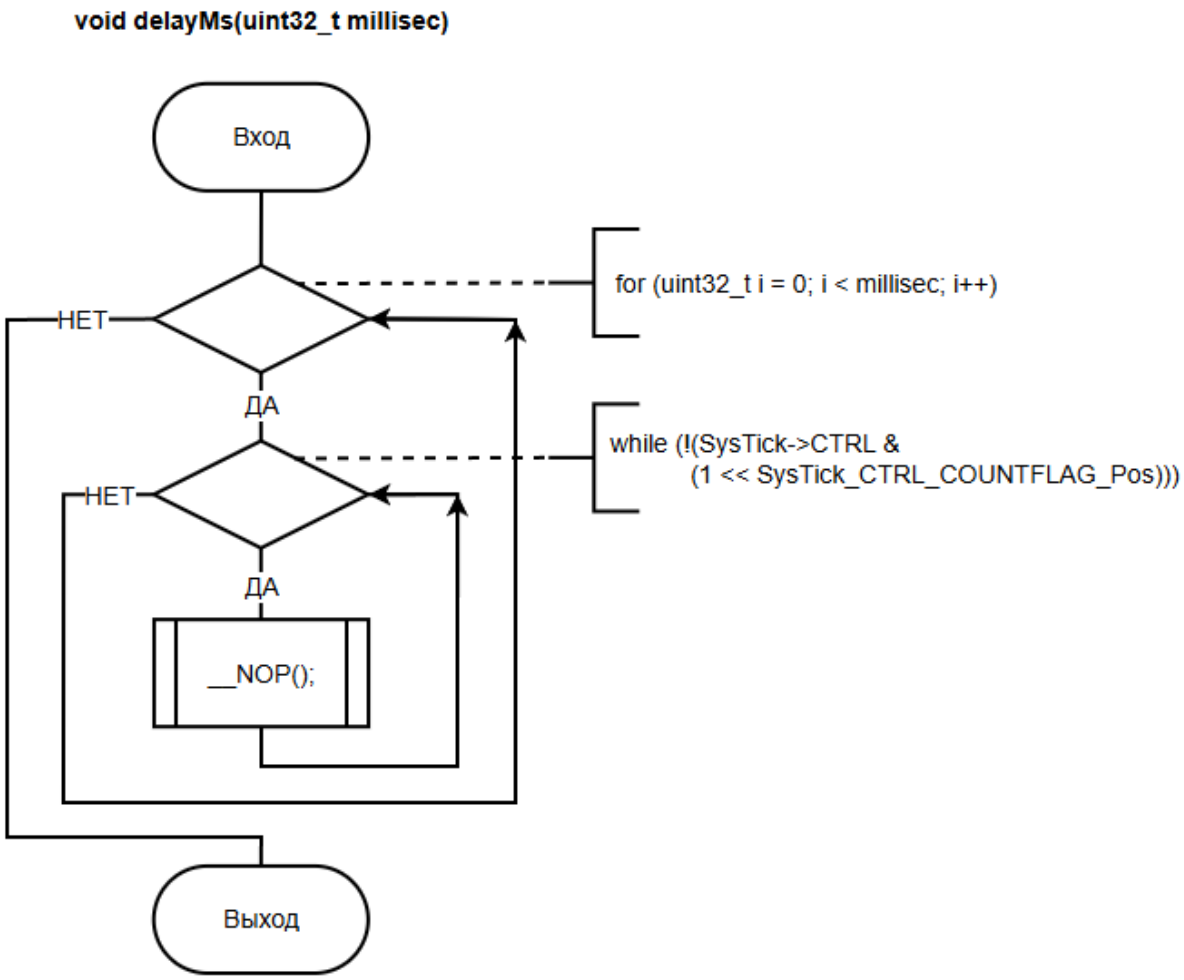


Рис. 4.1.3. Алгоритмическая блок-схема функции delayMs()

Перейдём к части работы с кнопкой. Кнопка является источником сигнала прерывания для МК, то есть такого электрического сигнала, который инициирует временную остановку основной функции **`main()`** и вызывает функцию-обработчик прерывания (Interrupt Service Routine – ISR), которая отвечает всем признакам обычной пользовательской функции, но при этом обладает одной важной особенностью – она может быть вызвана аппаратным, а не программным способом, то есть её исполнение зависит от определённых внешних или внутренних – относительно МК – событий и обстоятельств.

За работу прерываний в STM32 отвечает модуль процессора Nested Vector Interrupt Controller (NVIC). Использование прерываний в целом довольно простое, нужно выполнить три действия:

- Разрешить прерывания в базовом модуле NVIC для конкретной периферии;
- Разрешить интересующие прерывания в самом модуле периферии (некоторые модули внутри МК имеют несколько векторов [источников] прерываний);
- Прописать все необходимые функции-обработчики прерываний для каждого разрешённого события из NVIC.

Кнопки относятся к источникам так называемых внешних прерываний, то есть устройствам, которые коммутируют внешний сигнал на МК. Модуль внешних прерываний управляется собственными регистрами структуры EXTI.

Рассмотрим пример с кнопкой KEY (см. Рис. 3.4), распаянной прямо на плате Black Pill и подключенной к выводу PA0 МК. Исходя из Рис. 4.1.4 можно отметить, что кнопка на PA0 коммутирует на вход МК сигнал логического нуля (GND).

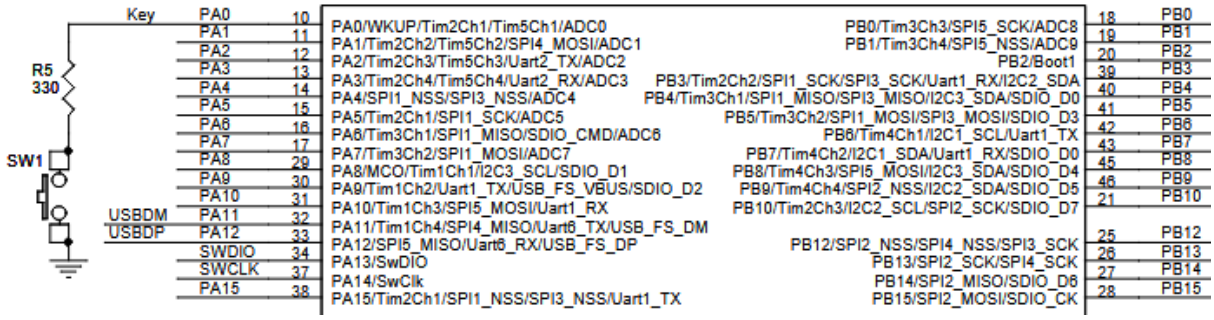


Рис. 4.1.4. Подключение кнопки SW1 к выводу PA0 МК

Для того, чтобы настроить вывод МК как вход сигнала внешнего прерывания, следует сначала настроить вывод МК на вход в регистре GPIO*x*_MODER. После этого нужно обратить внимание на Рис. 4.1.5, на котором приведено распределение линий прерываний EXTI*i* и их корреляция с выводами МК.

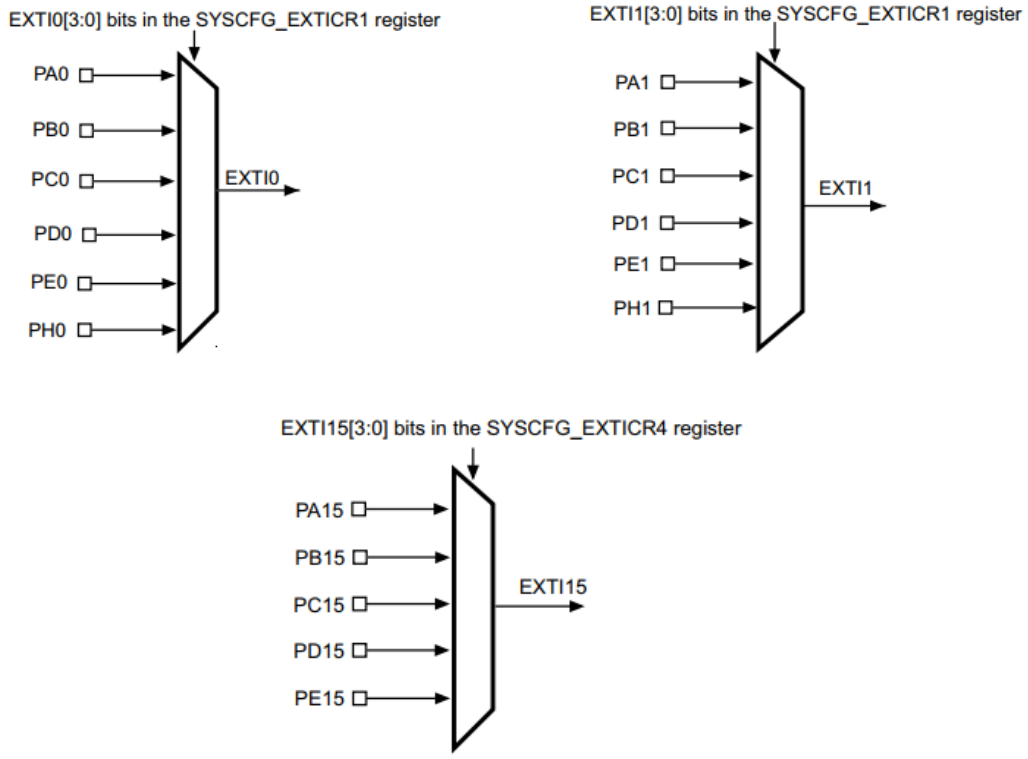


Рис. 4.1.5. Распределение линий внешних прерываний EXTI*i*

Из Рис. 4.1.5 видно, что всего у МК есть 16 линий внешних прерываний и на каждой линии может быть выбран только один вывод, например, на линии

EXTI0 может быть вывод 0 любого порта МК, но – только один (то есть настроить на линии EXTI0 выводы PD0 и PE0 – нельзя). Для того чтобы выбрать линию EXTI*i* следует обратиться к регистрам SYSCFG external interrupt configuration register *i* (SYSCFG_EXTICR*i*), комбинации из четырёх бит EXTI*i*[3:0] которого определяют какой вывод и порт будет настроен на соответствующей линии EXTI*i* МК: 0000 – PA*i*, 0001 – PB*i*, 0010 – PC*i*, 0011 – PD*i*, 0100 – PE*i*, 0110 – зарезервировано, 0111 – PH*i*.

Регистр **SYSCFG_EXTICR1**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI3[3:0]				EXTI2[3:0]				EXTI1[3:0]				EXTI0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Таких регистров 4 штуки, и они все одинаковые. Также следует отметить, что регистры сведены в массив, в котором элементы нумеруются начиная с нуля и на нулевой позиции стоит SYSCFG_EXTICR1 и далее располагаются регистр SYSCFG_EXTICR2, отвечающий за линии EXTI4-EXTI7, SYSCFG_EXTICR3, отвечающий за линии EXTI8-EXTI11 и SYSCFG_EXTICR3, отвечающий за линии EXTI12-EXTI15 (см. документ Reference manual). Кстати, для SYSCFG надо включить тактирование, и тактируется он от шины APB2.

Перейдём к рассмотрению регистров модуля EXTI, и начнём с регистра EXTI interrupt mask register (EXTI_IMR), который позволяет разрешить прерывание на конкретной линии (например, в качестве прерываний используются линии EXTI1, EXTI3 и EXTI4 – значит нужно выставить биты MR1, MR3 и MR4 регистра EXTI_IMR).

Регистр **EXTI_IMR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									MR22	MR21	Reserved		MR18	MR17	MR16
									rw	rw			rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Далее рассмотрим регистры, которые позволяют выбрать событие, по которому будет вызываться функция-обработчик прерывания ISR. Таких событий предоставляется на выбор два, а именно – нарастающий фронт (EXTI_RTSR) и падающий срез (EXTI_FTSR) сигнала на выводе.

Регистры **EXTI_RTSR** и **EXTI_FTSR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									TR22	TR21	Reserved		TR18	TR17	TR16
									rw	rw			rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Биты в обоих регистрах одинаковые, отличается только имя самого регистра. Кстати, можно настроить событие на срабатывание по любому изменению уровня (переход сигнала из 1-в-0 и из 0-в-1) просто выставив соответствующие биты в обоих регистрах.

Следующий регистр носит название EXTI pending register (EXTI_PR). Этот регистр позволяет узнать какие линии EXTI сработали и при входе в функцию-обработчик прерывания нужно сбросить флаг сработавшей линии. Сбрасываются биты в регистре EXTI_PR путём записи в них логической единицы (что следует из типа доступа к этим битам).

Регистр **EXTI_PR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									PR22	PR21	Reserved		PR18	PR17	PR16
									rc_w1	rc_w1			rc_w1	rc_w1	rc_w1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PR15	PR14	PR13	PR12	PR11	PR10	PR9	PR8	PR7	PR6	PR5	PR4	PR3	PR2	PR1	PR0
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

После инициализации и разрешения внешнего прерывания в модуле EXTI следует разрешить прерывание в модуле NVIC, делается это с помощью вызова функции **`NVIC_EnableIRQ(IRQn_Type IRQn)`**. Аргументом функции является вектор прерывания, их можно подсмотреть в файле **`stm32f401xc.h`**, который должен быть добавлен в проект. Эта функция является надстройкой над регистрами NVIC, работать с которыми не слишком удобно и именно поэтому пользователю дали возможность пользоваться функциями (эти функции расположены в файле **`core_cm4.h`**). Помимо функции разрешения прерывания есть ещё несколько:

- **`NVIC_DisableIRQ(IRQn_Type IRQn)`** – запрещение прерывания,
- **`NVIC_GetPendingIRQ(IRQn_Type IRQn)`** – узнать статус Pending прерывания,
- **`NVIC_SetPendingIRQ(IRQn_Type IRQn)`** – установить статус Pending прерывания,
- **`NVIC_ClearPendingIRQ(IRQn_Type IRQn)`** – сбросить статус Pending прерывания,
- **`NVIC_GetActive(IRQn_Type IRQn)`** – узнать статус Active прерывания,
- **`NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority)`** – установить приоритет прерывания,
- **`NVIC_GetPriority(IRQn_Type IRQn)`** – узнать приоритет прерывания.

Последний момент, который бы хотелось отметить про кнопку KEY (тоже самое касается и кнопки Button), которая приведена на Рис. 4.1.4, заключается в том, что если обратить внимание на способ подключения кнопки можно заметить, что когда кнопка нажата, на вход PA0 МК (или PB12 – в зависимости от кнопки) приходит логический ноль (GND), а когда кнопка не нажата, то на вход МК приходит неизвестно что, потому что на линии

появляется разрыв электрической цепи. Поэтому для корректной работы, следует сделать на выводе подключения кнопки МК внутреннюю подтяжку к питанию для того, чтобы при нажатии на кнопку обеспечить переход логического сигнала из 1 в 0. Для этого будет использоваться регистр GPIO*x*_PUPDR, а именно комбинации из двух бит PUPDR*i*[1:0] определяют какой тип подтяжки будет использоваться на выводе *i* порта *x* МК: 00 – без подтяжки, 01 – подтяжка к питанию (Pull-Up), 10 – подтяжка к земле (Pull-Down).

Регистр **GPIO*x*_PUPDR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Исходя из вышенаписанного про прерывания, приведём пример инициализации кнопки KEY на выводе PA0, изображённой на Рис. 4.1.4, как источника сигнала прерывания.

```
// настроить PA0 как входную ногу МК
GPIOA->MODER &= ~(3 << GPIO_MODER_MODE0_Pos);
// на PA0 включить внутренний pull-up
GPIOA->PUPDR |= (1 << GPIO_PUPDR_PUPD0_Pos);
// включить тактирование SysCfg
RCC->APB2ENR |= (1 << RCC_APB2ENR_SYSCFGEN_Pos);
// разрешить прерывания порта A
SYSCFG->EXTICR[0] = (0 << SYSCFG_EXRICR1_EXTI0_Pos);
// разрешить прерывание на выводе 0
EXTI->IMR |= (1 << EXTI_IMR_MR0_Pos);
// настроить прерывание на срез
EXTI->FTSR |= (1 << EXTI_FTSR_TR0_Pos);
// разрешить прерывания в NVIC, имя лежит в хедере stm32f401xc.h
NVIC_EnableIRQ(EXTI0_IRQn);
```

После этого остаётся только отметить, что в проекте должна появиться функция-обработчик прерывания по линии EXTI0:

```
void EXTI0_IRQHandler(void) {
    // сбросить флаг Pending события
    EXTI->PR = (1 << EXTI_PR_PR0_Pos);
    // запретить прерывание на выводе 0
    EXTI->IMR &= ~(1 << EXTI_IMR_MR0_Pos);
    // TODO
    // разрешить прерывание на выводе 0
    EXTI->IMR |= (1 << EXTI_IMR_MR0_Pos);
}
```

На месте строки `// TODO` должны появиться инструкции, являющиеся результатом нажатия на кнопку, подведённую к выводу PA0 МК. Имена функций для каждого вектора прерываний можно подсмотреть в стартап-скрипте **`startup_stm32f401ccux.s`**.

Работа 2. Реализация системы управления яркостью на основе ШИМ и обработки дискретных сигналов с подавлением дребезга

Электрическая схема к лабораторной работе приведена на Рис. 4.2.1.

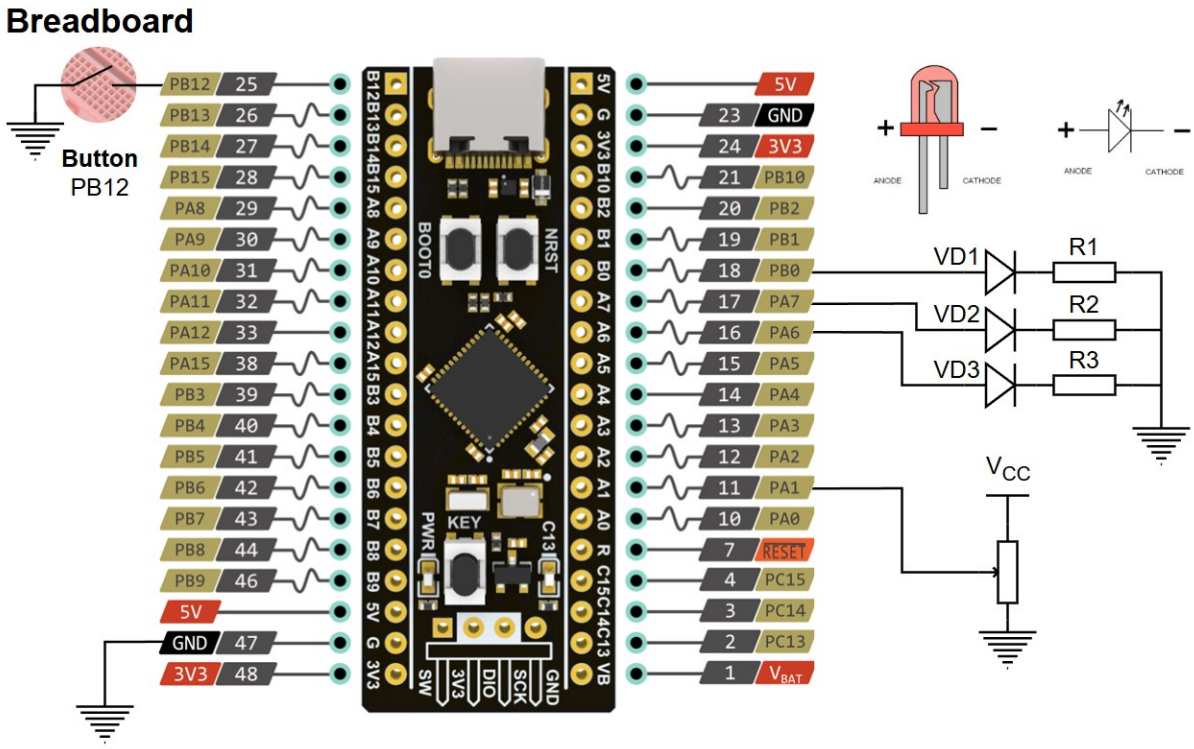


Рис. 4.2.1. Схема к лабораторной работе №2

К МК подключены 3 светодиода VD*i*, каждый подведён анодом к выводу МК. Кнопка Button, расположенная слева от платы Black Pill, подведена к выводу PB12. Также в работе принимает участие поворотный потенциометр VR2, подключенный к выводу PA1 МК.

Задание на работу №2

1. добавьте функционал гашения дребезга нажатия на кнопку Button с помощью таймера/счётчика TIM4 в код лабораторной работы №1;

2. напишите программу для управления яркостью свечения светодиодов VD*i* (ШИМ); яркость свечения задаётся с помощью потенциометра VR2, подключённого к каналу ADC1_IN1 (вывод PA1); выбор регулируемого светодиода осуществляется нажатием на кнопку Button (также с гашением дребезга).

Программа должна сохранять работоспособность при настройке частоты тактирования на HSI, HSE и PLL.

Рассмотрим более подробно то, как выглядит коммутация сигнала кнопкой. Пример такого процесса приведён на Рис. 4.2.2. Нетрудно видеть, что кнопка из приведённого примера коммутирует логический ноль, однако переход сигнала от уровня логической единицы к уровню логического нуля

происходит не мгновенно и занимает некоторое время. Этот эффект называется «дребезг» (bouncing) и считается общепринятым, что он должен быть подавлен, потому что из-за этого эффекта могут фиксироваться ложные нажатия кнопки. Подавление дребезга заключается в том, чтобы в момент первого изменения логического уровня сигнала на выводе внешнего прерывания МК запустить таймер, установленный на некоторое значение времени (например, 10…20 мс) и по истечению этого времени проверить состояние на выводе внешнего прерывания ещё раз и если на выводе всё ещё уровень напряжения, коммутируемый кнопкой, то в этот момент можно утверждать, что кнопка действительно была нажата.

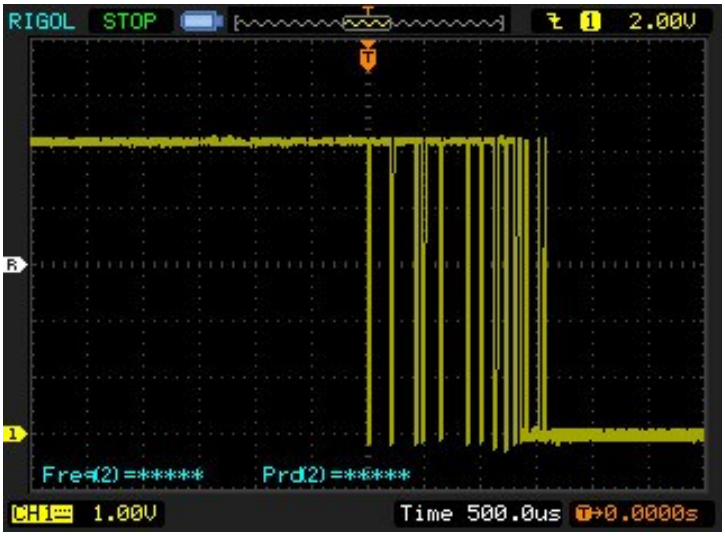


Рис. 4.2.2. Коммутация сигнала кнопкой

В данной работе ставятся три подзадачи:

1. Гашение дребезга кнопки при помощи таймера;
2. Выработка ШИМ сигнала на светодиоды при помощи таймера;
3. Получение аналогового сигнала с потенциометра и использование его в качестве задания коэффициента заполнения ШИМ сигнала.

В состав МК входит ещё один важный модуль – модуль таймеров. Ранее была приведена информация про системный таймер SysTick, который является частью процессора, но на самом деле таймеров есть гораздо больше и функционал их гораздо шире, например:

- Отсчет времени: если задача должна быть выполнена через известное время, можно однократно запустить таймер и выполнить необходимые действия по его переполнению;
- Периодические операции: если запустить таймер в непрерывном режиме, можно выполнять действия с фиксированной частотой;
- Реализовать системные часы: такую задачу решали с помощью SysTick Timer.
- Генерировать ШИМ-сигнал: таймер может управлять выводами GPIO для создания прямоугольного сигнала с управляемым коэффициентом заполнения;

- Определять частоту и длительность периодического сигнала на выводе: таймер может фиксировать значение счетчика в момент изменения напряжения на выводе;
- Синхронизировать работу других модулей: таймер может генерировать системные события, на которые можно настроить другие модули, например, измерение с помощью АЦП с заданной частотой.

Таймеры классифицируются по различным критериям, таким как функциональные возможности или битность. По битности таймеры подразделяются на 16- и 32-битные (см. Рис. 3.1 – таймер TIM9 является 16-битным, а таймер TIM2 – 32-битным). Выбирать таймер лучше исходя из поставленной задачи, то есть брать минимально подходящий по своему функционалу под конкретную задачу, а то вдруг в будущем потребуется таймер помощнее, а он оказался занят ещё в начале проекта под какую-то незначительную задачу.

Для использования любого из таймеров следует включить его тактирование в модуле RCC, при этом стоит учитывать, что разные таймеры могут быть привязаны к разным шинам (см. Рис. 3.1).

Алгоритм гашения дребезга кнопки приведён на Рис. 4.2.3.

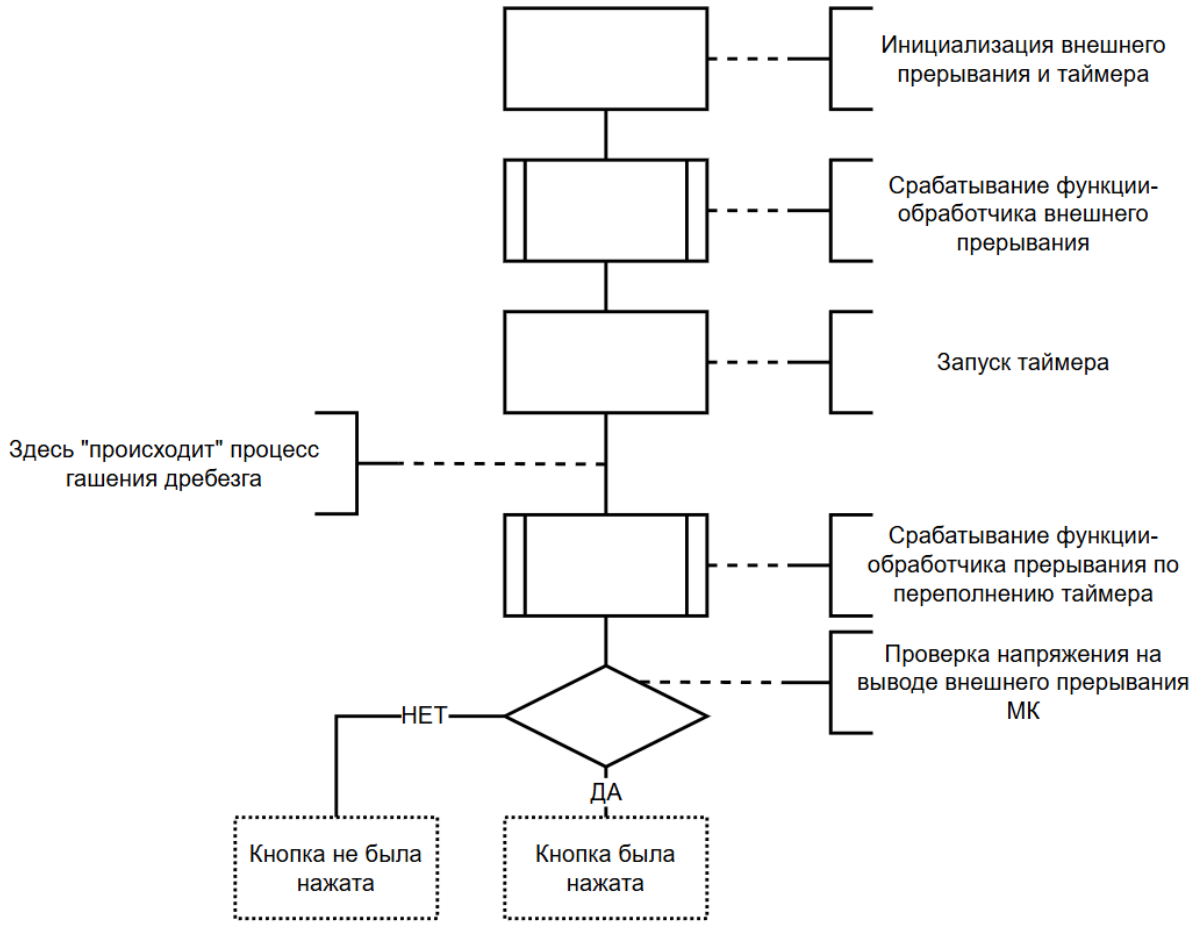


Рис. 4.2.3. Алгоритм гашения дребезга кнопки

Перейдём к рассмотрению регистров таймеров и начнём с временных регистров. Счётное значение таймера хранится в регистре TIMx counter (TIMx_CNT), который хранит либо 16-битное, либо 32-битное значение счёта таймера.

Регистр **TIMx_CNT**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNT[31:16] (depending on timers)															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Любой таймер работает на довольно высокой частоте и, соответственно, быстро считает. Чтобы замедлить частоту счёта имеется регистр предделителя TIMx prescaler (TIMx_PSC).

Регистр **TIMx_PSC**

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

В регистр предделителя кладётся число на единицу меньше желаемой величины (чтобы нельзя было задать 0), например, если таймер тактируется с частотой 84 МГц, то задав предделитель (8400 – 1), получим частоту счёта таймера 10 кГц.

Указав длительность одного такта (частоту таймера), следует указать сколько тактов нужно отсчитать. За это отвечает регистр TIMx auto-reload register (TIMx_ARR).

Регистр **TIMx_ARR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ARR[31:16] (depending on timers)															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

В регистр TIMx_ARR кладётся значение, которое будет считаться переполнением для счётного регистра таймера. Парой регистров – TIMx_PSC и TIMx_ARR – можно задать период счёта таймера согласно формуле:

$$T_{TIMER} = \frac{(PSC + 1) \cdot ARR}{f_{APB}}$$

Далее рассмотрим регистры настройки режимов. Регистр TIMx control register 1 (TIMx_CR1), а именно: бит ARPE управляет буферизацией ARR, если он выставлен, то значение ARR будет применяться только после события переполнения счётчика; бит OPM (когда выставлен) включает режим однократного счёта, то есть таймер сосчитает от начала до конца только один раз; бит URS позволяет настроить источник прерывания по событию переполнения, если он установлен, то только переполнение таймера вызывает

запрос прерывания, если он сброшен, то не только переполнение вызывает запрос прерывания; бит UDIS (когда выставлен) позволяет запретить обновление значений ARR и PSC; бит CEN (когда выставлен) запускает таймер.

Регистр **TIMx_CR1**

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CKD[1:0]		ARPE	CMS		DIR	OPM	URS	UDIS	CEN
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Регистр разрешения прерываний TIMx DMA/interrupt enable register (TIMx_DIER). Про то, что такое DMA здесь упоминания не будет, отметим лишь бит UIE, который в случае своего выставления разрешает прерывание по переполнению таймера.

Регистр **TIMx_DIER**

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	TDE	Res	CC4DE	CC3DE	CC2DE	CC1DE	UDE	Res.	TIE	Res	CC4IE	CC3IE	CC2IE	CC1IE	UIE
	rw		rw	rw	rw	rw	rw		rw		rw	rw	rw	rw	rw

Для работы с прерываниями нужно рассмотреть здесь ещё один регистр – TIMx status register (TIMx_SR). В нём находятся флаги прерываний, которые нужно сбрасывать в функции-обработчике прерываний ISR, например, бит UIF отвечает за запрос прерывания по переполнению таймера. Обратите внимание, что все биты в регистре TIMx_SR сбрасываются записью нуля, установка единицы игнорируется.

Регистр **TIMx_SR**

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CC4OF	CC3OF	CC2OF	CC1OF	Reserved		TIF	Res	CC4IF	CC3IF	CC2IF	CC1IF	UIF
			rc_w0	rc_w0	rc_w0	rc_w0			rc_w0		rc_w0	rc_w0	rc_w0	rc_w0	rc_w0

Приведём здесь пример инициализации таймера №2. Пусть частота тактирования МК настроена на HSE, 25 МГц.

```
TIM2->PSC = 25000 - 1; // частота счёта таймера 1 кГц
TIM2->ARR = 2; // период счёта таймера 2 мс
// событие обновления – только переполнение таймера
TIM2->CR1 |= (1 << TIM_CR1_URS_Pos);
// разрешить прерывания по переполнению (обновлению)
TIM2->DIER |= (1 << TIM_DIER_UIE_Pos);
NVIC_EnableIRQ(TIM2_IRQn); // разрешить прерывания в NVIC
```

Из кода, написанного выше, очевидно, что таймер №2 был настроен, но не был запущен. В случае задачи гашения дребезга запуск таймера логично производить тогда, когда срабатывает внешнее прерывание, поэтому в функции–обработчике внешнего прерывания должна появиться строка:

```
// запуск таймера в режим счёта
TIM2->CR1 |= (1 << TIM_CR1_CEN_Pos);
```

Поскольку разрешено прерывание по переполнению таймера №2 в проекте должна появиться функция-обработчик прерывания по этому событию:

```
void TIM2_IRQHandler(void) { // гашение дребезга
      TIM2->CR1 &= ~(1 << TIM_CR1_CEN_Pos); // остановить таймер
      // проверка флага произошедшего события
      if (TIM2->SR & (1 << TIM_SR_UIF_Pos)) {
            // очистка флага события
            TIM2->SR &= ~(1 << TIM_SR_UIF_Pos);
            // TODO
      }
}
```

На месте строки `// TODO` должны появиться инструкции, позволяющие проверить напряжение на выводе МК, к которому подведена кнопка. Для проверки напряжения на выводе МК, настроенном на вход, существует регистр только для чтения GPIO port input data register (GPIO*x*_IDR).

Регистр GPIO***x*_IDR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Результатом выполнения данной части работы является возможность обрабатывать нажатие на кнопку, в результате чего происходит переключение зажжённого светодиода.

Следующая часть работы состоит в применении ШИМ модуляции сигнала. ШИМ – это периодические прямоугольные импульсы постоянного периода и переменного коэффициента заполнения, как это показано на Рис. 4.2.4 (коэффициент заполнения – отношение длительности *Active* к периоду *T*).

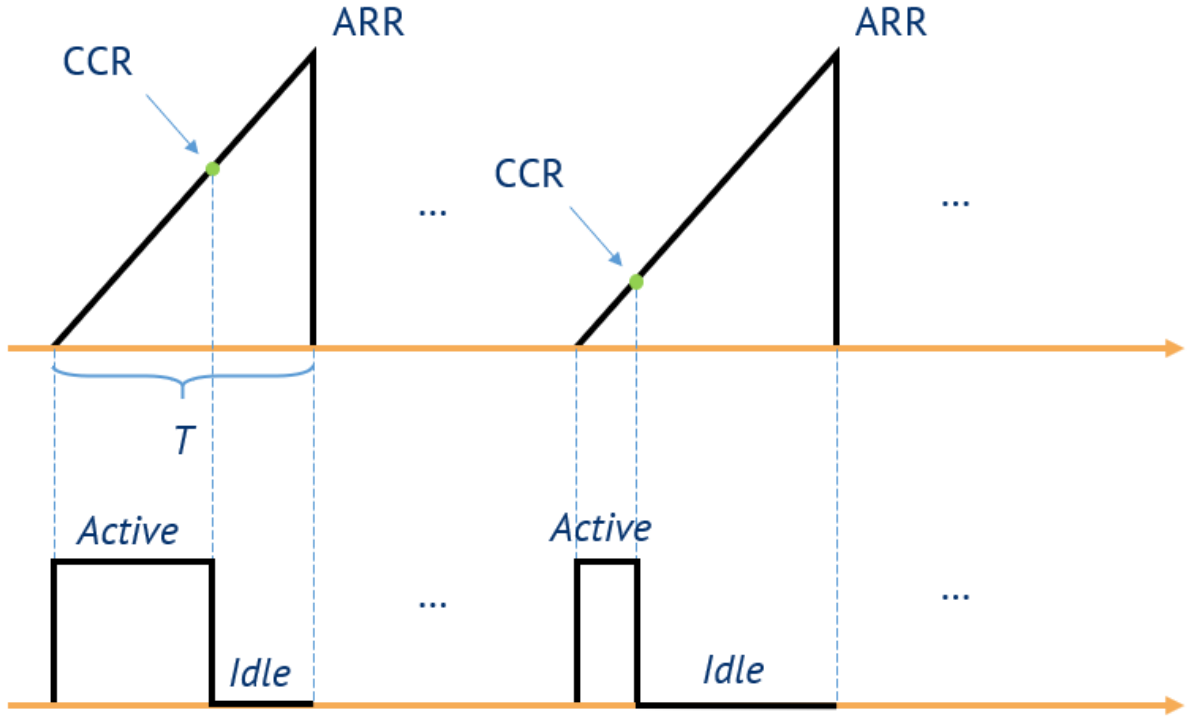


Рис. 4.2.4. Вверху – счёт таймера, внизу – визуализация ШИМ сигнала

Выработка ШИМ сигнала – это одно из множества применений для таймера. Исходя из Рис. 4.2.4 ARR будет определять период ШИМ, а отношение CCR к ARR – коэффициент заполнения. Ещё одним важным моментом является необходимость узнать к каким выводам подключены устройства, которыми нужно управлять, а именно – номер таймера и номер его канала. Исходя из Рис. 4.2.1, светодиоды подключены к выводам МК PB0, PA7 и PA6. Согласно документации, (datasheet) вывод PB0 является выводом канала 3 таймера №3, вывод PA7 – канала 2 таймера №3 и, наконец, вывод PA6 – канала 1 таймера №3. Базовая настройка таймера №3 ничем не отличается от настройки таймера №2, но добавляется ещё пара регистров.

Регистр TIMx capture/compare enable register (TIMx_CCER) позволяет включать и настраивать полярность соответствующих каналов, а именно: биты CC*n*E позволяют включить (когда установлен) канал номер *n* таймера, биты CC*n*P позволяют задать полярность канала *n* (если бит сброшен, это означает, что состояние Active выставляет высокий уровень напряжения, а состояние Idle – низкий, если бит установлен – наоборот).

Регистр **TIMx_CCER**

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CC4NP	Res.	CC4P	CC4E	CC3NP	Res.	CC3P	CC3E	CC2NP	Res.	CC2P	CC2E	CC1NP	Res.	CC1P	CC1E
rw		rw	rw	rw		rw	rw	rw		rw	rw	rw		rw	rw

Регистры TIMx capture/compare mode register 1 & 2 (TIMx_CCMR1 & TIMx_CCMR2) позволяют задавать настройки режимов работы, а именно: биты OCnM[2:0] позволяют выбрать режим работы при настройке канала на выход:

- 000 – вывод МК канала отключён, сравнение CNT и CCR происходит, но на выводе МК канала ничего не меняется;
- 001 – вывод МК канала перейдёт в состояние Active по совпадению CNT с CRR, переход в Idle не произойдёт;
- 010 – вывод МК канала перейдёт в состояние Idle по совпадению CNT с CRR, переход в Active не произойдёт;
- 011 – режим переключения, при каждом совпадении вывод МК канала переходит в противоположное состояние;
- 100 – вывод МК канала всегда в состоянии Idle;
- 101 – вывод МК канала всегда в состоянии Active;
- 110 – ШИМ, тип первый, пока CNT меньше CCR, на выводе будет состояние Active, когда больше – Idle;
- 111 – ШИМ, тип второй, пока CNT меньше CCR, на выводе будет состояние Idle, когда больше – Active.

Регистр **TIMx_CCMR1**

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2CE	OC2M[2:0]			OC2PE	OC2FE	CC2S[1:0]		OC1CE	OC1M[2:0]			OC1PE	OC1FE	CC1S[1:0]	
IC2F[3:0]				IC2PSC[1:0]				IC1F[3:0]				IC1PSC[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Осталось изучить регистр, в котором будет храниться значение CCR – TIM*x* capture/compare register (TIMx_CCR*n*), здесь *n* – это номер канала, которые есть у каждого конкретного таймера, например у таймера №2 есть 4 канала, но в работе будет использоваться 3 – по числу светодиодов.

Регистр **TIMx_CCR1**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CCR1[31:16] (depending on timers)															
rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR1[15:0]															
rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro

Следует отметить, что ШИМ – это альтернативная функция для выводов МК. Список альтернативных функций можно посмотреть в документе datasheet, глава pinout and pin description на конкретный МК. Ранее был рассмотрен регистр GPIO*x*_MODER, в описании которого было упомянуто, что если положить значение 2 в соответствующую позицию GPIO_MODER_MODE*i*_Pos, то вывод *i* порта МК будет настроен на альтернативную функцию. Самих этих функций довольно много, но, чтобы не обременять читателя необходимостью поисков, отметим, что каналы 1, 2 и 3 таймера №3 являются альтернативной функцией номер 2 для выводов МК (т.н. AF02). Для того чтобы выбрать альтернативную функцию, есть пара регистров GPIO*x*_AFRL и GPIO*x*_AFRH, AFRL отвечает за альтернативные функции выводов 0…7, а AFRH – за альтернативные функции выводов 8…15. При этом, в CubeIDE эта пара регистров сведена в массив, в котором элемент 0 – это AFRL, а элемент 1 – это AFRH. Таким образом, каждые 4 бита AFRL*i*[3:0] или AFRH*i*[3:0] позволяют выбрать альтернативную функцию для вывода *i* порта GPIO*x* МК, например 0 – AF00, 1 – AF01, 2 – AF02, 3 – AF03, …, 15 – AF15.

Регистр **GPIO*x*_AFRL**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRL7[3:0]				AFRL6[3:0]				AFRL5[3:0]				AFRL4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRL3[3:0]				AFRL2[3:0]				AFRL1[3:0]				AFRL0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Пример инициализации вывода PA6 МК и ШИМ на канале 1 таймера №3 приведён ниже.

```
// включение тактирования GPIOA
RCC->AHB1ENR |= (1 << RCC_AHB1ENR_GPIOAEN_Pos);
// настройка PA6 на альтернативную функцию
GPIOA->MODER |= (2 << GPIO_MODER_MODE6_Pos);
// выбор альтернативной функции AF2 для PA6
GPIOA->AFR[0] |= (2 << GPIO_AFRL_AFSEL6_Pos);
// включение тактирования таймера №3
RCC->APB1ENR |= (1 << RCC_APB1ENR_TIM3EN_Pos);
TIM3->PSC = 1000 - 1; // частота счёта таймера F_CPU / 1000
TIM3->ARR = 250; // период таймера 10 мс
```

```
// включение ШИМ режима на канале 1 таймера №3
TIM3->CCER |= (1 << TIM_CCER_CC1E_Pos);
// настройка режима ШИМ
TIM3->CCMR1 |= (6 << TIM_CCMR1_OC1M_Pos);
// запуск таймера в режим счёта
TIM3->CR1 |= (1 << TIM_CR1_CEN_Pos);
```

Для того, чтобы выработать ШИМ сигнал на вывод МК следует в коде прописать следующую строку.

```
// диод на выводе PA6 будет гореть в половину яркости
TIM3->CCR1 = 125;
```

Результатом выполнения данной части работы является написание цикла, в котором происходит изменение яркости всех трёх светодиодов одновременно.

Следующая часть работы состоит во взаимодействии с аналоговым устройством – потенциометром (резистор с переменным значением сопротивления). Схема потенциометра приведена на Рис. 4.2.5.

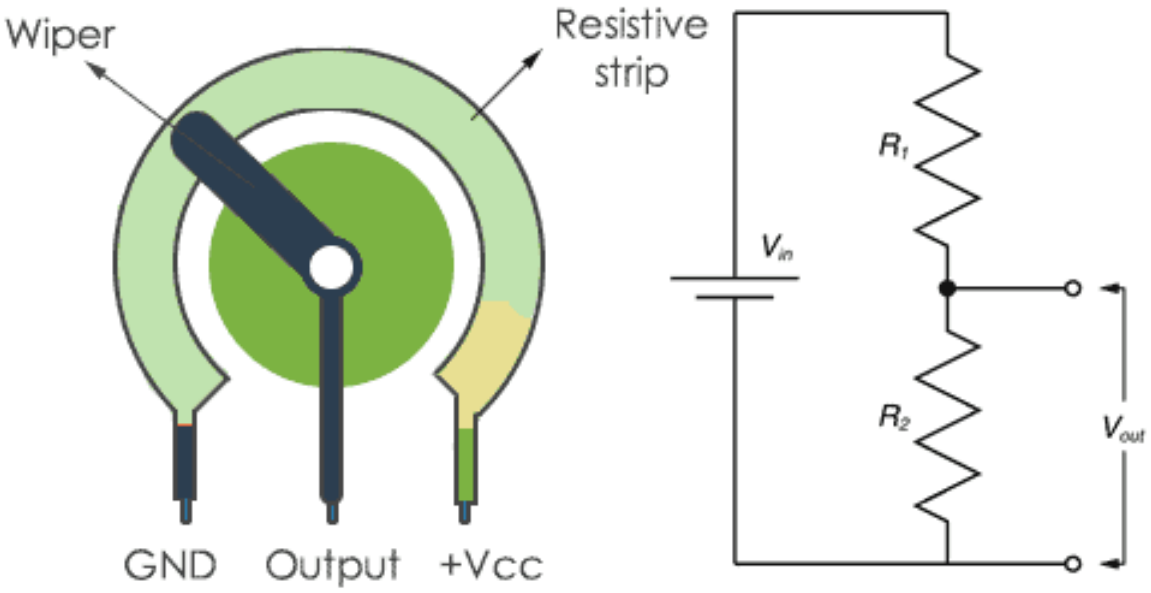


Рис. 4.2.5. Устройство потенциометра

Потенциометр вырабатывает аналоговое напряжение пропорциональное положению его скользящего контакта. МК не воспринимает аналоговый сигнал, для него существуют только логические уровни напряжения 0 и 1, поэтому внутри МК присутствует специальный модуль аналого-цифрового преобразования (АЦП), который позволяет представить аналоговое напряжение в виде диапазона чисел, в котором 0 – это одно крайнее положение рукоятки потенциометра (GND), а MAX – это противоположное крайнее положение рукоятки ($+V_{CC}$). Число MAX определяется разрядностью АЦП, в данном МК имеем 12-разрядный АЦП, то есть MAX = 4095.

Преобразование можно делать на выводах, которые являются каналами АЦП. В одном модуле АЦП обычно 16 каналов, в МК может быть один или несколько таких модулей. Измерение аналогового сигнала – это режим работы

выводов МК (см. регистр GPIO*x*_MODER, тип analog mode), поэтому его следует включить и настроить.

У модуля АЦП есть так называемый регистр состояния или статуса – ADC status register (ADC_SR[1]), в котором хранятся флаги различных событий, например – флаг EOC, который выставляется, когда АЦП заканчивает преобразование. Обратите внимание, что сбрасывается флаг записью логического нуля.

Регистр **ADC_SR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										OVR	STRT	JSTRT	JEOC	EOC	AWD
										rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0

Настройка АЦП, как и любого модуля, начинается с включения тактирования через RCC (см. Рис. 3.1). Помимо включения тактирования, следует включить ещё и сам модуль АЦП, это можно сделать через регистр ADC control register 2 (ADC_CR2), а именно – установкой бита ADON. Установка бита SWSTART запускает преобразование для обычной группы каналов. Группа каналов – это набор выводов ADC_IN*i* МК, на которые приходит аналоговый сигнал, подлежащий преобразованию.

Регистр **ADC_CR2**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
reserved	SWSTART	EXTEN		EXTSEL[3:0]				reserved	JSWSTART	JEXTEN		JEXTSEL[3:0]			
	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
reserved				ALIGN	EOCS	DDS	DMA	Reserved						CONT	ADON
				rw	rw	rw	rw							rw	rw

Группа каналов задаётся в регистрах ADC regular sequence register *i* (ADC_SQR*i*). Всего таких регистров 3, они содержат последовательности SQ1-SQ16 и L – длину последовательности.

Регистр **ADC_SQR1**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								L[3:0]				SQ16[4:1]			
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ16_0	SQ15[4:0]					SQ14[4:0]					SQ13[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw				rw	rw	rw	rw	rw

Регистр **ADC_SQR3**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		SQ6[4:0]					SQ5[4:0]					SQ4[4:1]			
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ4_0	SQ3[4:0]					SQ2[4:0]					SQ1[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

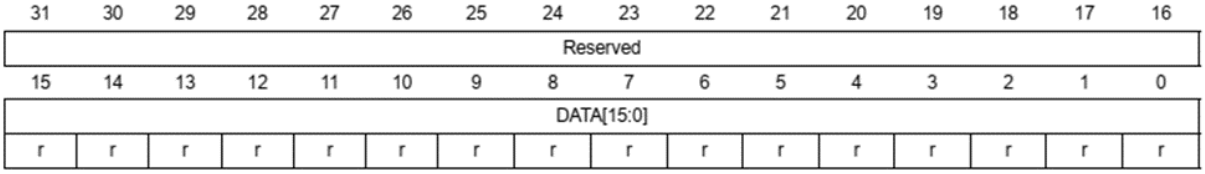
[1] У АЦП есть порядковый номер – ADC1, поэтому все его регистры записываются через ADC1->*REG*

В поля SQ*i* задаются элементы последовательности – номера каналов АЦП, сигнал на которых надо преобразовывать, причём *i* – это порядковый номер элемента последовательности преобразования. В поле L следует положить число на единицу меньше, чем количество измеряемых каналов. Например, если требуется читать сигнал с двух каналов – ADC_IN3 и ADC_IN7, настройка может выглядеть следующим образом:

```
ADC1->SQR1 = (1 << ADC_SQR1_L_Pos);
ADC1->SQR3 = (3 << ADC_SQR3_SQ1_Pos) | (7 << ADC_SQR3_SQ2_Pos);
```

Работа преобразователя заключается в запуске преобразования в нужный момент и сохранении результата из регистра данных ADC data register (ADC_DR).

Регистр **ADC_DR**



Далее приведём пример настройки АЦП для преобразования сигнала с одного канала ADC_IN0 (PA0).

```
uint16_t adcReadValue(void) {
	ADC1->CR2 |= (1 << ADC_CR2_SWSTART_Pos);
	while (!(ADC1->SR & (1 << ADC_SR_EOC_Pos))) {
		__NOP();
	}
	ADC1->SR &= ~(1 << ADC_SR_EOC_Pos);
	return ADC1->DR;
}

int main(void) {
	// TO DO
	// вывод PA0 является каналом ADC1_IN0
	RCC->APB2ENR |= (1 << RCC_APB2ENR_ADC1EN_Pos);
	// настраиваем PA0 на аналоговый режим
	GPIOA->MODER |= (3 << GPIO_MODER_MODE0_Pos);
	// включить модуль АЦП
	ADC1->CR2 |= (1 << ADC_CR2_ADON_Pos);
	// положить канал ADC_IN0 в последовательность измерений
	ADC1->SQR3 = (0 << ADC_SQR3_SQ1_Pos);
	for (;;) {
		// прочитать преобразованное значение
		uint16_t res = adcReadValue();
	}
}
```

Теперь разобраны все три части работы: гашение дребезга кнопки, запуск ШИМ и работа с АЦП, нужно только свести три части работы в одной программе.

_____Работа 3. Многоканальное аналого-цифровое преобразование с использованием DMA: чтение внутреннего датчика температуры и передача данных по UART

Электрическая схема к лабораторной работе приведена на Рис. 4.3.1.

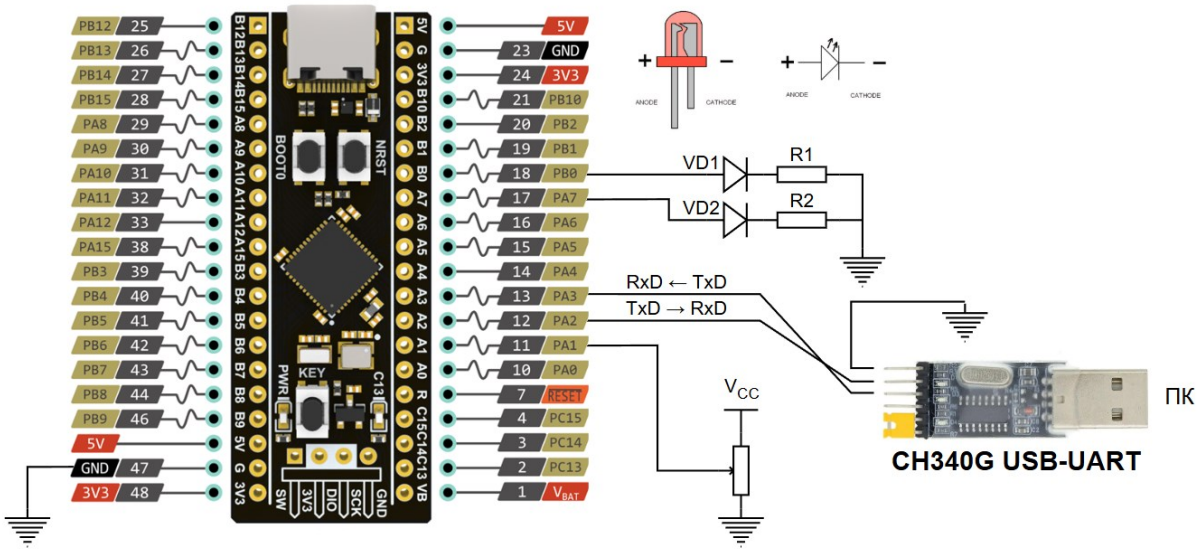


Рис. 4.3.1. Схема к лабораторной работе №3

К МК подключены 2 светодиода VD*i*, каждый подведён анодом к выводу МК. Поворотный потенциометр подключён к выводу PA1 МК (канал ADC1_IN1). Также в работе принимает участие встроенный в МК датчик температуры, подведённый к каналам АЦП ADC1_IN16 и ADC1_IN18. Справа на схеме изображён преобразователь USB-UART (название устройства – CH340G) и он может применяться для обмена данными между ПК и МК. Вывод TXD устройства подведён к выводу PA3 МК, вывод RXD подведён к выводу PA2 МК, вывод GND связан с GND МК.

Задание на работу №3

1. добавьте вывод значений с двух каналов АЦП ADC1_IN0 и ADC1_IN1 в терминал по протоколу UART в код лабораторной работы №2;

2. напишите программу для вывода трёх переменных в терминал по протоколу UART; тремя переменными являются фильтрованные значения с двух каналов АЦП, а именно: IN1 и IN16, считываемые с использованием DMA и значение температуры с внутреннего датчика, рассчитанное по формуле.

Программа должна сохранять работоспособность при настройке частоты тактирования на HSI, HSE и PLL.

В работе имеется несколько подзадач:

1. Передача данных (в частности – чисел) по протоколу UART на компьютер;
2. Преобразование значения с внутреннего датчика температуры МК;
3. Чтение нескольких каналов АЦП с использованием DMA.

Начнём с протокола UART. Протокол – это правила обмена информационными битами между двумя и более устройствами. Universal Synchronous Asynchronous Receiver-Transmitter USART или Универсальный Синхронно-Асинхронный Приёмо-Передатчик – это последовательный полнодуплексный протокол передачи данных между двумя устройствами – Master и Slave. В простом случае UART (без буквы S) – это средство связи, частота передачи данных по линиям которого определяется заранее и оба устройства её придерживаются. В этом случае протокол на аппаратном уровне состоит из трёх проводов – Rx (RxD), Tx (TxD) и GND. Частота передачи в этом случае ограничена единицами мегагерц.

USART – это периферийный модуль МК и следует помнить о необходимости включения его тактирования в модуле RCC (в данной работе используется USART2, см. Рис. 3.1). Также важно отметить, что функционал протокола USART – это альтернативная функция для пинов ввода-вывода МК (в данном случае – альтернативная функция AF07 выводов PA2 [USART2 TX] и PA3 [USART2 RX]), поэтому следует включить тактирование шины соответствующего порта и выбрать альтернативную функцию выводов МК, которая отвечает за работу протокола.

Рассмотрим регистры модуля USART и начнём с регистра состояний USART status register (USART_SR), в котором хранятся флаги. Бит TXE устанавливается аппаратно, когда данные были переданы из регистра данных USART_DR в сдвиговый регистр; бит сбрасывается аппаратно, когда пользователь производит запись в регистр USART_DR. Бит RXNE устанавливается аппаратно, когда содержимое сдвигового регистра было передано в регистр USART_DR; бит сбрасывается аппаратно, когда пользователь производит чтение из регистра USART_DR.

Регистр **USART_SR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NF	FE	PE
						rc_w0	rc_w0	r	rc_w0	rc_w0	r	r	r	r	r

Регистр данных USART_DR приведём здесь без картинки, так как это просто контейнер, в котором лежат биты – либо подлежащие передаче, либо которые были получены МК извне.

Регистр задания скорости тактирования приёмо-передачи данных USART baudrate register (USART_BRR).

Регистр **USART_BRR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIV_Mantissa[11:0]												DIV_Fraction[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

USART_BRR состоит из двух частей, одна из которых названа DIV_Mantissa (целая часть числа), а вторая – DIV_Fraction (дробная часть числа). В этот регистр кладётся значение предделителя, которое и определяет частоту обмена данными по протоколу. Рассмотрим пример. Пусть частота шины МК, от которой тактируется модуль протокола составляет 42 МГц. Требуется настроить частоту передачи по протоколу на 19200 бод. Согласно формуле из Reference manual, имеем следующее:

$$baud = \frac{f_{CK}}{8 * (2 - OVER8) * USART_BRR},$$

где $baud$ - желаемая скорость протокола, f_{CK} - частота тактирования модуля USART, $OVER8$- передискретизация (по умолчанию – значение 0). В нашем случае:

$$USART_BRR = \frac{42000000}{8 * (2 - 0) * 19200} \approx 136{,}72,$$

тогда число 136 помещается в часть DIV_Mantissa целиком, а вот остаток – 0,72 умножается на 16 и кладётся в часть DIV_Fraction (4 бита, $2^4 = 16$).

Регистр настройки USART control register 1 (USART_CR1).

Регистр **USART_CR1**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
rw	Res.	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Бит UE отвечает за включение модуля USART, когда этот бит выставлен – соответствующий модуль будет включён, если сброшен – аппаратные выводы и предделитель USART не активен. Бит M определяет размер посылки: 0 – 1 старт-бит, 8 бит данных и *n* стоп-битов, 1 – 9 бит данных. Биты TE и RE разрешают работу передатчика и приёмника USART, соответственно.

Согласно документации на STM32, для настройки протокола на стороне МК, необходимо выполнить следующую последовательность шагов:

1. Включить модуль протокола выставив бит UE в регистре USART_CR1.
2. Выбрать число передаваемых бит данных в бите M в регистре USART_CR1.
3. Выбрать число стоповых бит выставив значение битов STOP[1:0] в регистре USART_CR2 (по умолчанию – 1 стоп-бит).
4. Настроить частоту работы протокола в регистре настройки бодрейта USART_BRR (частоту можно настроить любую, но обычно настраивают на одну из стандартных частот от 1.2 кбод до 3 Мбод).
5. Выставить бит TE регистра USART_CR1 для того, чтобы включить режим передачи.
6. Записать в регистр данных USART_DR последовательность бит, подлежащих передаче. При этом сбросится бит TXE регистра состояния USART_SR.

7. В случае использования модуля протокола в режиме однократной передачи – дождаться установки бита TXE в регистре USART_SR, что означает что регистр данных очищен и последовательность бит передана.

Приведём пример настройки USART. Пусть у нас есть МК с модулем USART2, который тактируется от шины APB1, частота которой 25 МГц. Согласно документации, USART2_TX и USART2_RX – это альтернативная функция AF07 выводов МК PA2 и PA3, соответственно. Настраивать будем только передачу данных на скорости 115200 бод.

```
// включить тактирование GPIOA
RCC->AHB1ENR |= (1 << RCC_AHB1ENR_GPIOAEN_Pos);
// включить тактирование USART2
RCC->APB1ENR |= (1 << RCC_APB1ENR_USART2EN_Pos);
// настроить выводы PA2 и PA3 на альтернативную функцию
GPIOA->MODER |=
(2 << GPIO_MODER_MODE2_Pos) | (2 << GPIO_MODER_MODE3_Pos);
// выбрать альтернативную функцию AF07 для PA2 и PA3
GPIOA->AFR[0] |=
(7 << GPIO_AFRL_AFSEL2_Pos) | (7 << GPIO_AFRL_AFSEL3_Pos);
// настроить частоту 115200 бод
USART2->BRR =
(13 << USART_BRR_DIV_Mantissa_Pos) |
(8 << USART_BRR_DIV_Fraction_Pos);
// включить модуль USART2 и передачу данных
USART2->CR1 |= (1 << USART_CR1_UE_Pos) | (1 << USART_CR1_TE_Pos);
```

Для того, чтобы МК передал один байт по USART можно написать функцию для передачи байта.

```
void uartTransmitByte(uint8_t byte) {
	USART2->DR = byte; // положить байт в регистр
	while (!(USART2->SR & USART_SR_TXE)) {
		__NOP(); // дождаться установки флага передачи
	} // теперь байт передан
}
```

Чтобы воспользоваться написанной функцией нужно вызвать её следующим образом.

```
// вывод буквы a в терминал ('a' – ASCII код букв a)
uartTransmitByte('a');
// вывод цифры 6 (плюс шесть к ASCII коду цифры ноль) в терминал
uartTransmitByte(6 + '0');
```

Чтобы увидеть передаваемые с МК данные, нужно на компьютере запустить любую программу-терминал, выбрать COM-порт, который создаётся при подключении USB-UART преобразователя и выбрать частоту, из которой был рассчитан USART_BRR.

Осталось дописать функции для передачи чисел и может быть ещё слов и на этом часть работы с протоколом USART закончена.

Перейдём к части с чтением нескольких каналов АЦП. В МК есть встроенный датчик температуры (см. Рис. 3.1, левый нижний угол). Поддерживаемый диапазон температур от –40 до +125 °C с точностью до ±1.5 °C. Датчик температуры подключён к каналам АЦП ADC1_IN16 и ADC1_IN18. Для вывода датчика температуры из спящего режима следует выставить бит TSVREF в регистре ADC common control register (ADC_CCR).

Регистр **ADC_CCR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								TSVREFE	VBATE	Reserved				ADCPRE	
								rw	rw					rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															

Далее можно прочитать значение V_{SENSE} из регистра ADC_DR и рассчитать текущую температуру по формуле:

$$Temperature\ (in\ Celcium) = \left(\frac{V_{SENSE} - V_{25}}{Avg_{slope}}\right) + 25,$$

где V_{SENSE} – пропорциональное значению температуры напряжение на канале датчика температуры АЦП, V_{25} – значение V_{SENSE} при 25 °C (согласно документации равно 0.76 В), Avg_{slope} – табличное значение угла наклона кривой изменения температуры (согласно документации 2.5 мВ/°C). Например, пусть используется 12-битный результат преобразования АЦП, значит максимальное значение АЦП составляет 4095 при напряжении на канале 3.3 В, а значит $V_{SENSE} = 0.76$ В ≈ 943 ед. АЦП.

Поскольку потенциометр всё ещё подключен к ADC1_IN1 (см. Рис. 4.3.1), возникает задача чтения нескольких каналов АЦП. В регистре ADC control register 1 (ADC_CR1) следует включить режим сканирования каналов АЦП SCAN.

Регистр **ADC_CR1**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved					OVRIE	RES		AWDEN	JAWDEN	Reserved					
					rw	rw	rw	rw	rw						

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DISCNUM[2:0]			JDISCEN	DISCEN	JAUTO	AWDSGL	SCAN	JEOCIE	AWDIE	EOCIE	AWDCH[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

После этого в регистр ADC1_SQR3 следует положить каналы 16 и 1 в позиции SQ1 и SQ2, а в регистре ADC1_SQR1 изменить значение L выставив его в 1 (число L равно числу измеряемых каналов минус один).

Согласно документации, также рекомендуется увеличить время преобразования канала, на котором расположен датчик температуры (можно увеличить время преобразования для всех используемых каналов). Делается это с помощью регистра ADC sample time register i ADC_SMPRi, в которых каждые тройка битов SMPi[2:0] отвечает за время преобразования каждого члена последовательности из SQR, а именно: 000 – 3 цикла на преобразование, 001 – 15, 010 – 28, 011 – 56, 100 – 84, 101 – 112, 110 – 144, 111 – 480.

Регистр **ADC_SMPR1**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved					SMP18[2:0]			SMP17[2:0]			SMP16[2:0]			SMP15[2:1]	
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMP15_0	SMP14[2:0]			SMP13[2:0]			SMP12[2:0]			SMP11[2:0]			SMP10[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Теперь необходимо забирать данные из единственного регистра данных ADC_DR, в который будет помещаться преобразованное значение с каждого канала АЦП по очереди в пределах обычной (регулярной) группы. И вот тут возникает сложность с тем, что необходимо успевать эти данные забирать между преобразованиями, а это не так просто, потому что преобразования проходят очень быстро. В помощь нам будет модуль DMA.

Перейдём к части работы с DMA – direct memory access. DMA – это инструмент копирования данных в обход ЦП. Копирование возможно в следующих направлениях:

- из памяти в периферию;
- из периферии в память;
- из памяти в память.

В МК у каждого модуля DMA есть несколько потоков (Stream), в каждом из которых есть несколько каналов (Channel). Если необходимо включить передачу нужно начать с определения подходящего канала. Для этого в Reference Manual представлены таблички. У каждого потока можно выбрать только один канал, поэтому периферийные модули дублируются в разных потоках, чтобы уменьшить шанс конфликта, когда все подходящие потоки заняты. Таблица для DMA2 приведена на Рис. 4.3.2.

Peripheral requests	Stream 0	Stream 1	Stream 2	Stream 3	Stream 4	Stream 5	Stream 6	Stream 7
Channel 0	ADC1	-	-	-	ADC1	-	TIM1_CH1 TIM1_CH2 TIM1_CH3	-
Channel 1	-	-	-	-	-	-	-	-
Channel 2	-	-	-	-	-	-	-	-
Channel 3	SPI1_RX	-	SPI1_RX	SPI1_TX	-	SPI1_TX	-	-
Channel 4	SPI4_RX	SPI4_TX	USART1_RX	SDIO	-	USART1_RX	SDIO	USART1_TX
Channel 5	-	USART6_RX	USART6_RX	SPI4_RX	SPI4_TX	-	USART6_TX	USART6_TX
Channel 6	TIM1_TRIG	TIM1_CH1	TIM1_CH2	TIM1_CH1	TIM1_CH4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	-
Channel 7	-	-	-	-	-	-	-	-

Рис. 4.3.2. Каналы и потоки модуля DMA2

Рис. 4.3.2 описывает именно DMA2, потому что у него в канале 0 потока 0 есть искомый модуль ADC1.

Для настройки DMA первым делом надо включить тактирование соответствующего модуля в регистре RCC. Далее следует выбрать канал, задать адреса чтения/записи, направление движения данных и их размер, а также – указать количество передаваемых пакетов данных. Все изменения в настройках DMA производятся при отключённом модуле. Регистр DMA stream *x* configuration register (DMA_S*x*CR).

Регистр **DMA_S*x*CR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reser-ved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Следует отметить то, как обращаться к регистрам DMA. Например, чтобы отключить DMA2 Stream 0, запись будет выглядеть следующим образом:

```
DMA2_Stream0->CR &= ~(1 << DMA_SxCR_EN_Pos);
```

То есть у каждого модуля DMA и каждого потока есть своя собственная структура.

В регистре DMA_SxCR нужно сделать следующее:

- С помощью битов CHSEL[2:0] выбрать канал – число, соответствующее номеру канала Channel.
- Указать приоритет данного канала DMA на шине данных (биты PL[1:0] – priority level). Допустимые значения: 00 – низкий, 01 – средний, 10 – высокий, 11 – очень высокий.
- Задать размер данных для периферии (биты PSIZE[1:0]) и памяти (биты MSIZE[1:0]). Есть варианты: 00 – 8 бит, 01 – 16 бит и 11 – 32 бита.
- Указать будет ли инкрементироваться адрес данных в памяти (бит MINC) и/или периферии (бит PINC). В обоих случаях 1 означает, что адрес инкрементируется, а 0 – адрес постоянный. Изменение адреса нужно, например, чтобы собрать множество данных в буфере для последующей обработки.
- Выставить бит CIRC если требуется включить циклический режим работы – в этом случае настройки DMA будут автоматически применяться после завершения очередной передачи.
- Если используется циклический режим следует выставить разрешение прерывания по окончанию передачи – бит TCIE.

Количество передаваемых данных указывается в регистре DMA stream *x* number of data register (DMA_S*x*NDTR). Сюда кладется 16-битное значение желаемых передач данных. После передачи каждого элемента счетчик уменьшается на единицу. Когда число дойдет до нуля передача будет считаться завершенной.

Регистр **DMA_S*x*NDTR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NDT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Адрес периферийного модуля откуда/куда передаются данные указывается в регистре DMA stream *x* peripheral address register (DMA_S*x*PAR). Сюда необходимо положить адрес регистра, который можно увидеть, например, в документации в описании любого модуля периферии. Но гораздо удобнее писать имя регистра, так как CMSIS, по сути, и связывает программные структуры с адресами в памяти МК. Таким образом запись адреса периферии может выглядеть как:

```
// поток 0 DMA2, адрес периферии – регистр данных ADC1
DMA2_Stream0->PAR = (uint32_t) &ADC1->DR;
```

Адрес данных указывается аналогично, но здесь это просто указатель на переменную. Не забывайте, что имя массива это уже указатель на начало памяти, выделенной под этот массив. Таким образом запись адреса данных может выглядеть как:

```
// поток 0 DMA2, адрес данных – массив adcResult
DMA2_Stream0->M0AR = (uint32_t) adcResult;
```

Все флаги статусов доступны в регистрах статусов – DMA low interrupt status register (DMA_LISR) и DMA high interrupt status register (DMA_HISR). Оба эти регистра – только для чтения.

Регистр **DMA_LISR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				TCIF3	HTIF3	TEIF3	DMEIF3	Reserved	FEIF3	TCIF2	HTIF2	TEIF2	DMEIF2	Reserved	FEIF2
r	r	r	r	r	r	r	r		r	r	r	r	r		r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				TCIF1	HTIF1	TEIF1	DMEIF1	Reserved	FEIF1	TCIF0	HTIF0	TEIF0	DMEIF0	Reserved	FEIF0
r	r	r	r	r	r	r	r		r	r	r	r	r		r

Чтобы иметь возможность сбросить тот или иной статус есть ещё два регистра – DMA low interrupt flag clear register (DMA_LIFCR) и DMA high interrupt flag clear register (DMA_HIFCR), например биты CTCIF*x* – индикаторы того, что поток *x* закончил передачу. Флаги сбрасываются запись логической единицы.

Регистр **DMA_LIFCR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CTCIF3	CHTIF3	CTEIF3	CDMEIF3	Reserved	CFEIF3	CTCIF2	CHTIF2	CTEIF2	CDMEIF2	Reserved	CFEIF2
				w	w	w	w		w	w	w	w	w		w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				CTCIF1	CHTIF1	CTEIF1	CDMEIF1	Reserved	CFEIF1	CTCIF0	CHTIF0	CTEIF0	CDMEIF0	Reserved	CFEIF0
				w	w	w	w		w	w	w	w	w		w

Если разрешено прерывание, например по окончанию передачи TCIE, нужно написать соответствующую функцию-обработчик прерывания, в

которой, помимо всего прочего, будет происходить сброс флага CTCIF*x*, где *x* – номер канала, регистра DMA*i*_*n*IFCR, где *i* – порядковый номер DMA, *n* – выбор регистра – Low или High.

Настройка DMA2 (поток 0, канал 0) на работу «периферия → память», где периферией выступает регистр данных ADC1 (12-битный результат преобразования АЦП, 2 измеряемых канала в регулярной группе), а памятью – 16-битный массив из двух элементов в программном коде описана ниже.

```
// включить тактирование DMA2
RCC->AHB1ENR |= RCC_AHB1ENR_DMA2EN;
DMA2_Stream0->CR &= ~(1 << DMA_SxCR_EN_Pos); // выключить DMA2
// DMA2, Stream 0, Channel 0, приоритет – очень высокий,
// Инкремент адреса памяти, 16 бит данных для передачи,
// Циклический режим, прерывание по завершению передачи
DMA2_Stream0->CR |=
(0 << DMA_SxCR_CHSEL_Pos) | (3 << DMA_SxCR_PL_Pos) |
(1 << DMA_SxCR_MINC_Pos) |
(1 << DMA_SxCR_MSIZE_Pos) | (1 << DMA_SxCR_PSIZE_Pos) |
(1 << DMA_SxCR_CIRC_Pos) | (1 << DMA_SxCR_TCIE_Pos);
DMA2_Stream0->NDTR = 2; // 2 переменных по 16 бит
DMA2_Stream0->PAR = (uint32_t) &ADC1->DR; // периферия – ADC1
DMA2_Stream0->M0AR = (uint32_t) adcResult; // память
NVIC_EnableIRQ(DMA2_Stream0_IRQn);
DMA2_Stream0->CR |= (1 << DMA_SxCR_EN_Pos); // включить DMA2
```

DMA может использоваться почти с любым периферийным модулем. Но работа может довольно сильно отличаться от модуля к модулю. Поэтому для обеспечения работы необходимо читать разделы документации по работе с DMA в описании интересующего модуля.

В общих словах алгоритм работы можно описать следующим образом:

- настройка DMA: для этого нужно выполнить необходимые настройки, описанные выше;
- настройка модуля с разрешением работы DMA в нужных режимах;
- запуск модуля DMA;
- запуск модуля, если именно он инициирует передачу данных.

Для обеспечения работы АЦП с DMA необходимо установить биты DMA и DDS в регистре CR2 модуля АЦП (см. Работу 2).

В этом случае по выполнению каждого преобразования будет сгенерирован запрос DMA и данные будут скопированы в соответствии с настройками DMA. После выполнения заданного количества преобразований (NDTR в DMA), программа перейдёт к функции-обработчику прерываний, в которой нужно будет сбросить флаг прерывания и снова запустить DMA.

После этого остаётся передать результаты преобразований всех каналов, лежащих в массиве по UART в терминал на ПК.